

Advanced Learning Algorithms

Neural Network - Neurons and the Brain

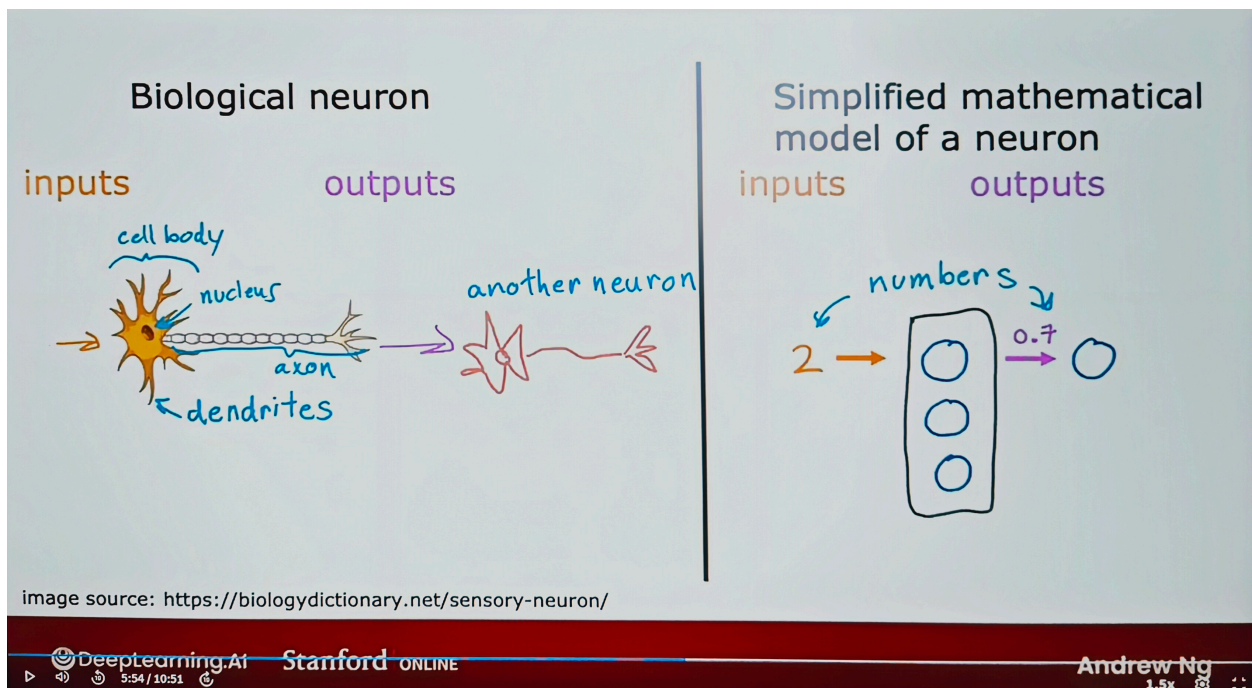
Initial motivation was to mimic the human brain.

Eg - Handwritten Digit recognition. Use

Deep Learning is much similar or Modern Neural Networks.

Evolved from speech recognition to image recognition to NLP or Natural Language Processing.

How we artificially mimic a neuron is, we create a model that mimics the functionality, i.e., it takes multiple inputs in a calculation and gives an output.



Still Human Brain functionality is yet to be known. Can't blindly mimic.

Why interest in Neural Networks now?

Neural networks are picking up now with rise of digitalization, amount of available data has increased.



Introduction to Neural Networks and Deep Learning

1. Historical Background

- Neural networks were originally motivated by the idea of mimicking how the human brain learns and thinks.
- Early work on neural networks began in the **1950s**, but the field went through periods of decline and resurgence:

- **1980s–1990s:** Gained traction with applications like **handwritten digit recognition** (used for reading postal codes and bank checks).
- **Late 1990s:** Interest declined again.
- **2005 Onwards:** Resurgence, rebranded as **deep learning**.
- The term "**deep learning**" became more popular than "neural networks" due to its appealing name.
- Since then, neural networks have revolutionized many fields.

2. Applications of Neural Networks

- **Speech recognition:** Major advancements due to deep learning.
- **Computer vision:** Breakthrough moment in **2012** with the **ImageNet** competition.
- **Natural Language Processing (NLP):** Improvements in text understanding.
- **Other domains:** Climate change, medical imaging, online advertising, product recommendations, etc.

3. Biological Basis vs. Artificial Neural Networks

- Although neural networks were inspired by the **human brain**, they have **evolved independently**.
- **Structure of a biological neuron:**
 - **Dendrites:** Receive signals.
 - **Cell body (nucleus):** Processes information.
 - **Axon:** Sends signals to other neurons.
- **Artificial Neural Networks (ANNs) vs. Biological Neurons:**
 - ANNs are **mathematical models** inspired by biological neurons.
 - ANNs consist of **nodes (neurons)** that take input, process data, and produce an output.

- While the biological inspiration remains, **modern ANNs do not directly mimic brain function.**

4. Growth of Neural Networks – Why Now?

- **Factors contributing to recent success:**

1. **Increased Data Availability:**

- The **rise of digitalization**, mobile phones, and the internet led to a **data explosion**.
- Traditional algorithms (like **linear regression** and **logistic regression**) struggled to utilize large datasets effectively.

2. **Performance Scaling:**

- Traditional algorithms plateaued with increasing data.
- **Neural networks**, especially **deep networks**, improved performance as more data was fed into them.

3. **Hardware Advancements:**

- **GPUs (Graphics Processing Units)**, originally designed for computer graphics, enabled faster neural network training.

5. How Neural Networks Work (High-Level Overview)

- **Basic structure:**

- **Inputs** (numbers).
- **Processing units (neurons)** that apply mathematical operations.
- **Outputs**, which serve as inputs for the next layer.

- **Deep Learning vs. Traditional Machine Learning:**

- Traditional ML algorithms struggle with large datasets.
- **Deep learning scales better** with increasing data, leading to breakthroughs in applications.

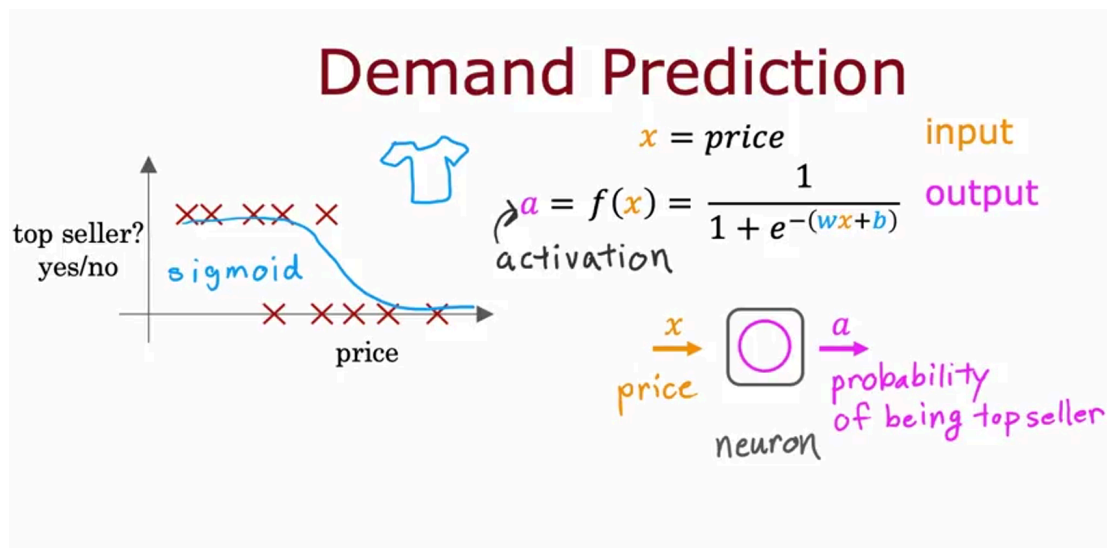
Eg - Demand Prediction

demand prediction refers to the process of estimating whether a product (e.g., a T-shirt) will be a **top seller** based on various factors

Understanding Neural Networks: Structure and Function

1. Example: Predicting Top-Selling Products

- **Use Case:** Predicting whether a T-shirt will be a top seller based on its features.
- **Input Feature (x):** Price of the T-shirt.
- **Model Used:** Logistic Regression.
 - Logistic regression applies a **sigmoid function**:
$$a = \frac{1}{1 + e^{-(wx + b)}}$$
 - **Activation (a):** Output probability of the T-shirt being a top seller.
 - Activation is a term from **neuroscience**, referring to how strongly a neuron fires.



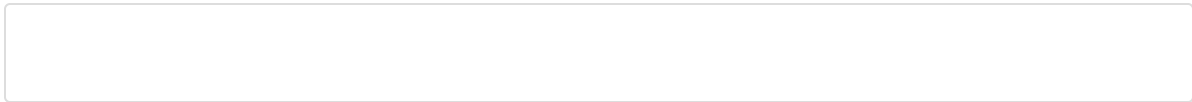
2. Neural Networks as an Extension of Logistic Regression

- A **single neuron** in an artificial neural network functions like a small **computer**:
 - It takes **inputs** (e.g., T-shirt price).
 - Performs **calculations** (e.g., logistic regression).
 - Produces **output** (e.g., probability of the T-shirt being a top seller).
- **Scaling Up**: Instead of a single neuron, a **network** of neurons can be used to handle more complex problems.

3. Multi-Feature Demand Prediction

- Instead of just price, a **better model** considers multiple factors:
 - **Price**
 - **Shipping cost**
 - **Marketing efforts**
 - **Material quality**
- These factors influence:
 - **Affordability** (depends on price & shipping cost).

- **Awareness** (depends on marketing).
- **Perceived quality** (depends on price & material quality).



Neural Network Model for Demand Prediction

- **Step 1:** Three neurons compute intermediate predictions:
 - **Affordability neuron** → Uses price & shipping cost.
 - **Awareness neuron** → Uses marketing data.
 - **Perceived quality neuron** → Uses price & material quality.
- **Step 2:** These three outputs (activations) are passed to a **final neuron** that predicts the probability of the T-shirt being a top seller.

4. Understanding Layers in Neural Networks

- **Input Layer:** Accepts raw input features (e.g., price, shipping cost).
- **Hidden Layer:** Processes intermediate activations (e.g., affordability, awareness, perceived quality).
- **Output Layer:** Produces the final result (e.g., probability of being a top seller).
- The term "**hidden layer**" comes from the fact that intermediate activations are not directly observable in the training dataset.

5. Fully Connected Networks

- In a real neural network, every neuron in a hidden layer receives inputs from **all neurons in the previous layer**.
- This allows the model to **learn which features are most relevant** instead of relying on manual feature selection.

6. Neural Networks Learn Their Own Features

- Traditional **feature engineering** requires manual selection of important features.
- **Neural networks automatically learn relevant features** through training.
- Example: Instead of manually computing **affordability**, the network learns it based on patterns in data.

7. Multi-Layer Networks (Deep Learning)

- **Adding more hidden layers** allows the network to learn increasingly **abstract representations**.
- Example of a deeper network:
 - **First hidden layer** → Extracts basic features.
 - **Second hidden layer** → Combines features into more complex patterns.
 - **Final hidden layer** → Produces refined features that make classification easier.

8. Neural Network Architecture Decisions

- Key architectural decisions:
 - **Number of hidden layers.**
 - **Number of neurons per layer.**
- The choice of architecture **affects model performance**.
- Deeper networks can learn more complex patterns but may require more data and computational power.

9. Multilayer Perceptron (MLP)

- A neural network with multiple hidden layers is called a **Multilayer Perceptron (MLP)**.
- MLP is a **feedforward network**, meaning information moves in one direction (from input to output).

Summary

- **Neural networks generalize logistic regression by stacking multiple neurons together.**
 - **They automatically learn features from data, eliminating the need for manual feature engineering.**
 - **Adding multiple hidden layers enables deep learning, allowing for more complex pattern recognition.**
 - **Choosing the right number of layers and neurons is critical for performance.**
-

OR

Introduction to Neural Networks

- Neural networks are used for prediction tasks, such as demand prediction
- Example application: Predicting if a T-shirt will be a top seller
- Practical use: Helps retailers plan inventory levels and marketing campaigns

From Logistic Regression to Neurons

- Input feature (x) such as price is fed into the learning algorithm
- Output prediction uses sigmoid function: $1/(1+e^{(-wx+b)})$
- Output termed as "activation" (a) - from neuroscience terminology
- Logistic regression unit can be viewed as a simplified model of a brain neuron
- A neuron is like a tiny computer that:

- Takes inputs (one or more numbers)
- Performs computation
- Outputs prediction (probability)

Building a Neural Network

- Neural networks combine multiple neurons wired together
- Example: T-shirt top seller prediction with four features:
 - Price
 - Shipping costs
 - Marketing amount
 - Material quality

Neural Network Structure

- **Hidden Layer Neurons:**
 - First neuron: Estimates affordability (based on price and shipping)
 - Second neuron: Estimates awareness (based on marketing)
 - Third neuron: Estimates perceived quality (based on price and material)
- **Output Layer:**
 - Takes the three estimates as inputs
 - Outputs probability of being a top seller

Neural Network Terminology

- **Layer:** Group of neurons taking similar inputs and outputting numbers together
- **Input Layer:** Original features (vector x)
- **Hidden Layer:** Middle layer(s) between input and output
- **Output Layer:** Final layer producing the prediction

- **Activations:** Output values from neurons (e.g., affordability, awareness, quality)

Practical Implementation

- In practice, each neuron in a layer connects to all outputs from previous layer
- Neural network learns which connections are important
- Neural network is simplified as:
 - Input layer: Vector of features
 - Hidden layer: Processes inputs and produces activation vector
 - Output layer: Processes hidden layer activations and produces final prediction

Why Hidden Layers Are "Hidden"

- In training data, you observe inputs (x) and outputs (y)
- You don't observe the correct values for intermediate concepts (affordability, etc.)
- These intermediate values are "hidden" in the training set

Neural Networks as Automatic Feature Engineering

- Neural networks can be viewed as logistic regression with learned features
- Instead of manual feature engineering, networks learn their own features
- This makes neural networks extremely powerful learning algorithms

More Complex Neural Network Architectures

- Neural networks can have multiple hidden layers
- Examples:
 - Network with two hidden layers: Input → Hidden Layer 1 → Hidden Layer 2 → Output

- Network with three hidden layers: Input → Hidden Layer 1 → Hidden Layer 2 → Hidden Layer 3 → Output
- Architecture decisions:
 - How many hidden layers to use
 - How many neurons per hidden layer
- These decisions impact learning algorithm performance
- Also called "multilayer perceptron" in some literature

Conclusion

- Neural networks process inputs through layers to produce predictions
 - Each layer transforms its inputs into new representations
 - The power comes from automatic feature learning
 - Architecture choices are important for performance
-

Neural Networks in Computer Vision: Face Recognition Example

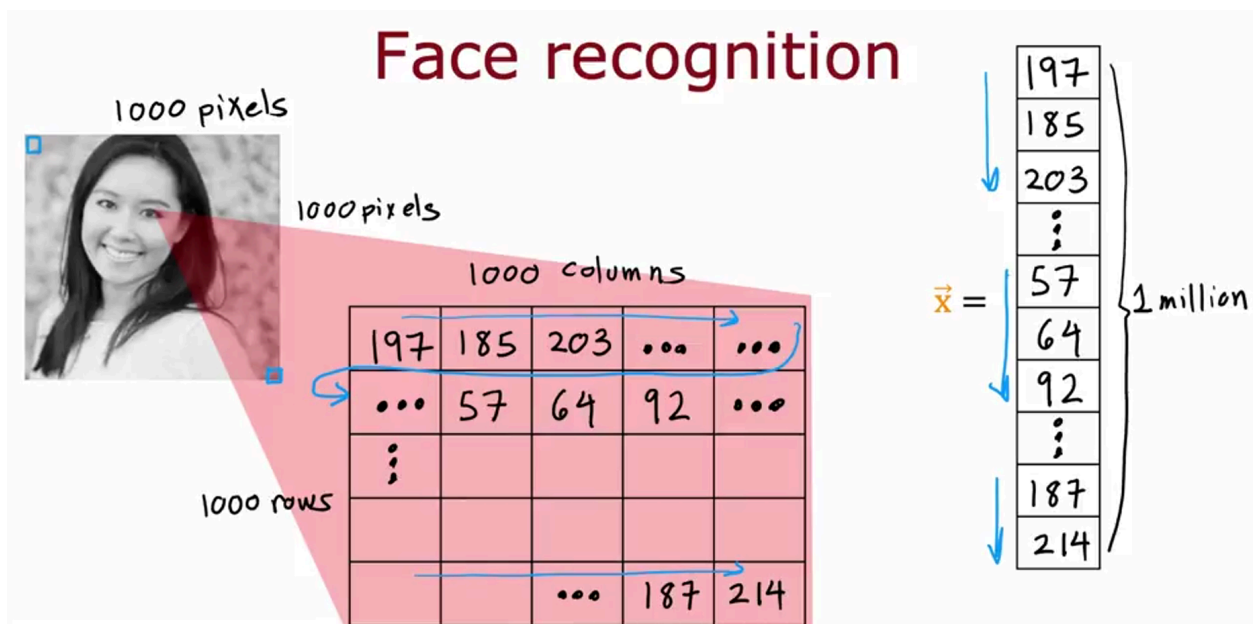
1. Introduction to Computer Vision with Neural Networks

- Neural networks can be used for **face recognition** by training them to identify a person in an image.
- **Example task:** Given an image of a face, predict the **identity** of the person.

Image Representation in a Computer

- Images are stored as **pixel grids**.

- A **1000 × 1000** pixel image is represented as a **matrix of pixel intensity values**.
- Each pixel has a **brightness value** ranging from **0 to 255**.
- Example:
 - Top-left pixel: **197**
 - Adjacent pixel: **185**
 - Bottom-right pixel: **214**
- **Flattening the Image:** The matrix is converted into a **vector of one million** pixel intensity values.

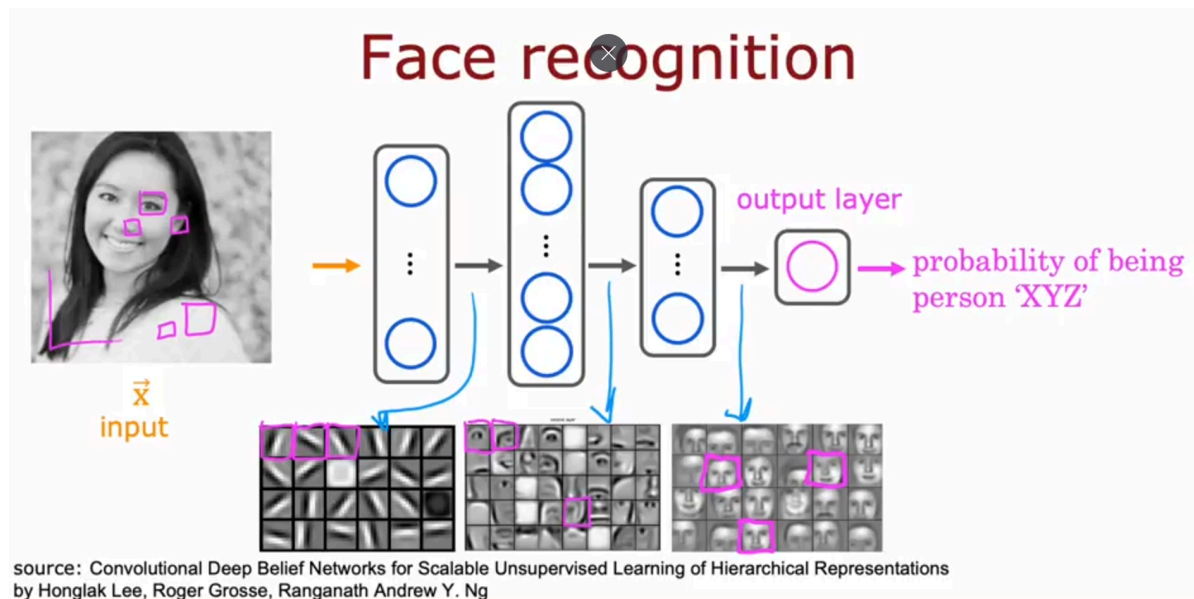


2. Neural Network Architecture for Face Recognition

- **Input:** A vector of pixel values.
- **Hidden Layers:**
 - Extract **features** from the image.
 - Each layer refines the understanding of the image.
- **Output Layer:** Predicts the **identity** of the person.

3. Understanding Feature Detection in Neural Networks

- Neural networks learn **hierarchical representations**:
 - **First hidden layer**: Detects **edges** (e.g., vertical/horizontal lines).
 - **Second hidden layer**: Detects **parts of the face** (e.g., eyes, nose, ears).
 - **Third hidden layer**: Detects **larger facial structures**.
 - **Final layer**: Recognizes **who the person is** based on learned features.



Visualization of Learned Features

- **First Layer**:
 - Detects **basic features** like **lines and edges**.
- **Second Layer**:
 - Detects **eyes, noses, and other facial features**.
- **Third Layer**:

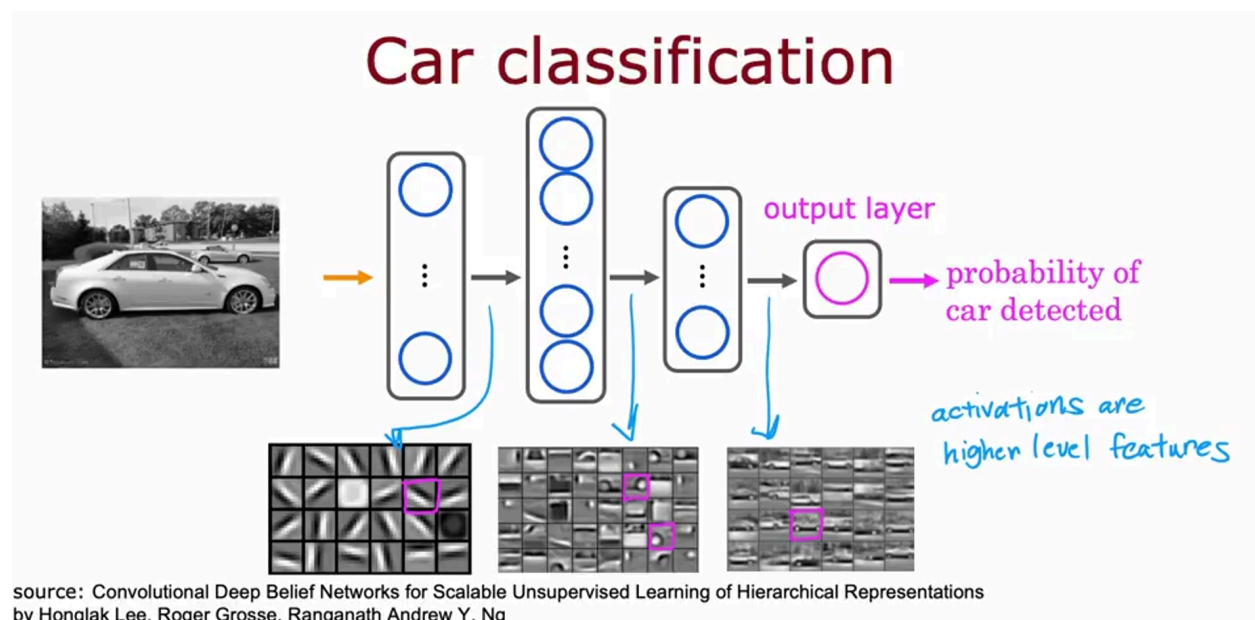
- Detects **overall face shapes**.
- **Final Layer:**
 - Identifies the **specific person** in the image.

Self-Learning Ability

- The network **automatically** discovers important features from data.
- **No manual feature engineering** is needed.

4. Neural Networks for Different Image Recognition Tasks

- If trained on **car images**, the network:
 - Learns **edges** in the first layer.
 - Detects **car parts** in the second layer.
 - Recognizes **full cars** in the third layer.
- The same **learning algorithm** adapts to different **datasets**.



5. Practical Application: Handwritten Digit Recognition

- Upcoming implementation will demonstrate a neural network applied to **handwritten digit recognition**.
 - Transition from **concepts** to **mathematical implementation**.
-

Summary

- **Neural networks in computer vision extract hierarchical features from images.**
 - **They learn edge detection, object parts, and complex patterns layer by layer.**
 - **The same neural network structure can be used for different image tasks (e.g., face recognition, car detection).**
 - **Upcoming lessons will cover the mathematical foundations and implementation of neural networks.**
-

fundamental building block of modern neural networks: **layers of neurons**. It covers how to construct a layer, the computations involved, and how layers interact to form a complete neural network.

Introduction

- **Purpose of Layers:**
 - Layers are the core building blocks of neural networks.
 - Understanding how a layer works enables the construction of larger, more complex networks.
- **Example Context:**
 - Demand prediction for a T-shirt:
 - Input features (e.g., price, shipping cost, marketing, material quality).
 - Hidden layer with 3 neurons.
 - Output layer with 1 neuron producing the final prediction.

Structure of a Neural Network

1. Input Layer:

- Takes in raw input features (e.g., price, shipping cost).
- Denoted as **Layer 0**.

2. Hidden Layer:

- Processes inputs from the input layer.
- Consists of multiple neurons performing computations.
- Example: Hidden layer with 3 neurons.

3. Output Layer:

- Processes outputs from the hidden layer.
- Produces the final prediction (e.g., probability or binary classification).

Zooming into the Hidden Layer

Inputs to the Hidden Layer

- The hidden layer receives 4 input features (e.g., price, shipping cost, marketing, material quality).
- These inputs are processed by each neuron in the hidden layer.

Neuron Computation

1. Each Neuron as a Logistic Regression Unit:

- Each neuron implements a logistic regression function.
- Formula for a single neuron: $a = g(w \cdot x + b)$

$$a = g(w \cdot x + b)$$

Where:

- w : Weight vector for the neuron.
- x : Input vector (features).

- b : Bias term.
- $g(z) = \frac{1}{1+e^{-z}}$: Sigmoid function.

2. Parameters for Each Neuron:

- Each neuron has its own weights (w) and bias (b).
- Example for three neurons in the hidden layer:
 - Neuron 1: w_1, b_1
 - Neuron 2: w_2, b_2
 - Neuron 3: w_3, b_3

3. Example Computations for Each Neuron:

- **Neuron 1:** $a_1 = g(w_1 \cdot x + b_1)$

$$a_1 = g(w_1 \cdot x + b_1)$$

Example result: $a_1 = 0.3$

Interpretation: A 30% chance that affordability is high based on input features.

- **Neuron 2:** $a_2 = g(w_2 \cdot x + b_2)$

$$a_2 = g(w_2 \cdot x + b_2)$$

Example result: $a_2 = 0.7$

Interpretation: A 70% chance that potential buyers are aware of the T-shirt.

- **Neuron 3:** $a_3 = g(w_3 \cdot x + b_3)$

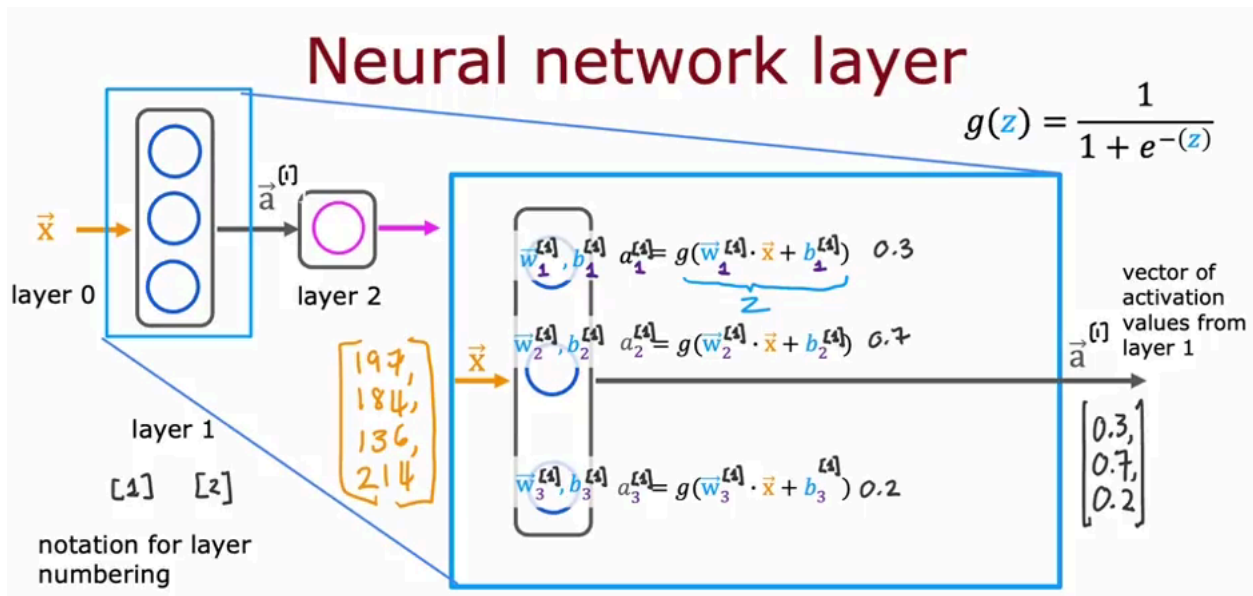
$$a_3 = g(w_3 \cdot x + b_3)$$

Example result: $a_3 = 0.2$

Interpretation: A 20% chance that perceived quality is high.

4. Output of the Hidden Layer:

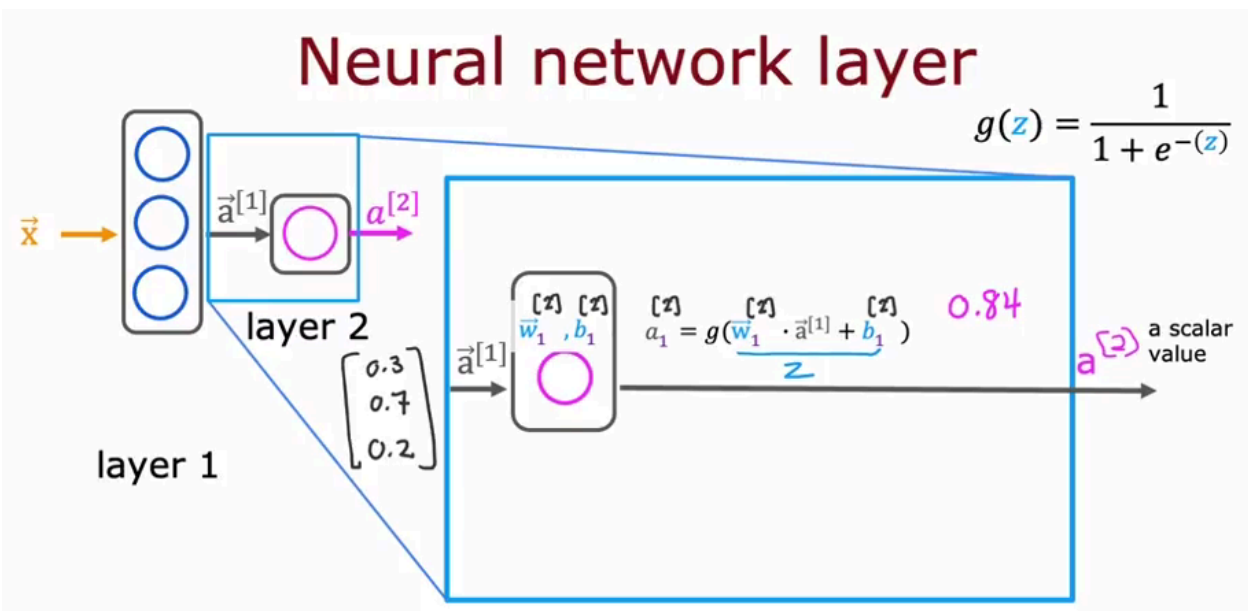
- The hidden layer outputs an activation vector: $a = [a_1, a_2, a_3] = [0.3, 0.7, 0.2]$
- This vector becomes the input to the next layer (output layer).



Notation for Layers and Parameters

Layer Indexing:

- Layers are indexed sequentially:
 - Input layer: Layer 0.
 - Hidden layers: Layer 1, Layer 2, etc.
 - Output layer: Final layer.
- Example:
 - Hidden layer is Layer 1.
 - Output layer is Layer 2.



Parameter Notation:

- Weights and biases for neurons in each layer are indexed with superscripts and subscripts to distinguish them across layers and neurons.
- Example for Layer 1 (hidden layer):
 - Neuron 1 parameters: w_1, b_1
 - Neuron 2 parameters: w_2, b_2
- Example for Layer 2 (output layer): w_1, b_1

Activation Values Notation:

- Activations for each layer are denoted as $a[l]$, where l is the layer number.
- Example:
 - Activations for hidden layer (Layer 1): $a = [a_1, a_2, a_3]$
 - where each activation corresponds to one neuron in Layer 1.
 - Activation for output layer (Layer 2): a

a

Zooming into the Output Layer

Inputs to the Output Layer

- The output layer receives the activation vector from the hidden layer ($a = [0.3, 0.7, 0.2]$).

Output Neuron Computation

- The single output neuron computes its activation as follows:

$$z = w \cdot a + b$$

z

$=$

w

$\cdot$

a

$+$

b

- Apply sigmoid function to compute activation value:

$$a = g(z)$$

a

$=$

g

$($

z

$)$

- Example result:

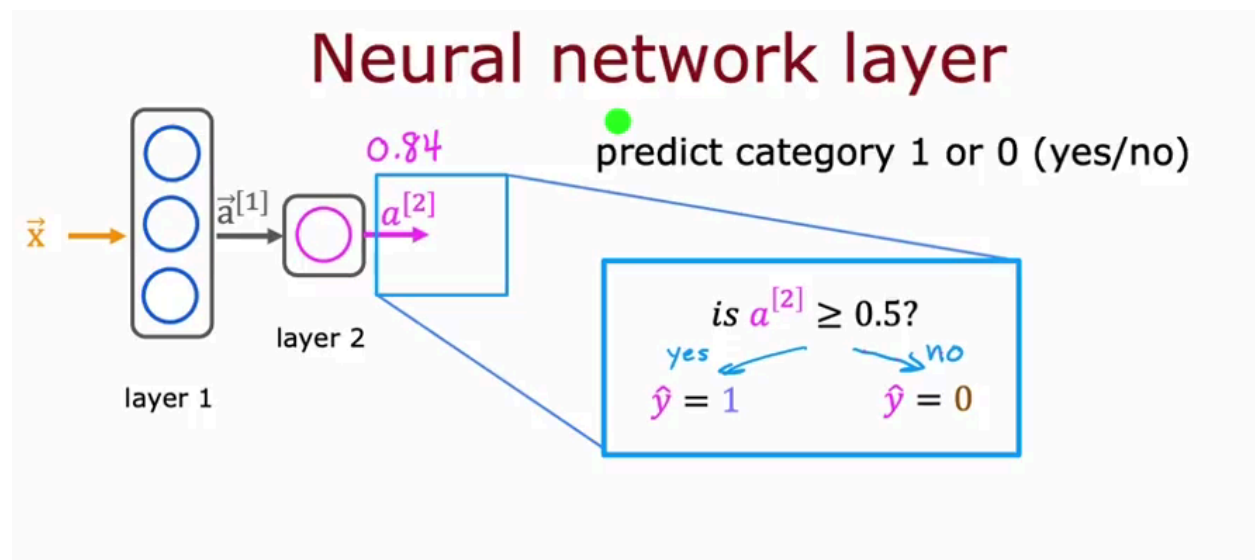
$$a = 0.84$$

a

$=0.84$

Final Prediction

- The output ($a=0.84$) represents the probability of being a top seller.
- Optional step: Thresholding at 0.5 for binary classification:
 - If $a > 0.5$ → Predict $\hat{y}=1$ (top seller).
 - If $a < 0.5$ → Predict $\hat{y}=0$ (not top seller).



Summary of Neural Network Computations

- Each layer processes input vectors using logistic regression units to compute activations.
- Activations are passed from one layer to another until reaching the final output.
- Final output can be either:
 - A probability (e.g., probability of being a top seller).

- A binary classification after thresholding at some value (e.g., threshold at 0.5).

Key Takeaways

- Neural networks consist of layers where each neuron performs computations based on weights and biases.
- Layers transform inputs into new representations through activations.
- Notation conventions help track parameters and activations across layers ($w[l], b[l], z[l], a[l]$).
- The final prediction can be used directly as probabilities or converted into binary classifications.

Next Steps

With this foundation in understanding layers and their computations, future topics will explore more complex neural network architectures with multiple layers and advanced models capable of solving diverse problems.

This session expands on the concept of neural network layers to construct a **more complex neural network**. It introduces a 4-layer network (excluding the input layer) and explains the computations within each layer, focusing on **notation, layer interactions, and activation functions**.

Structure of the Neural Network

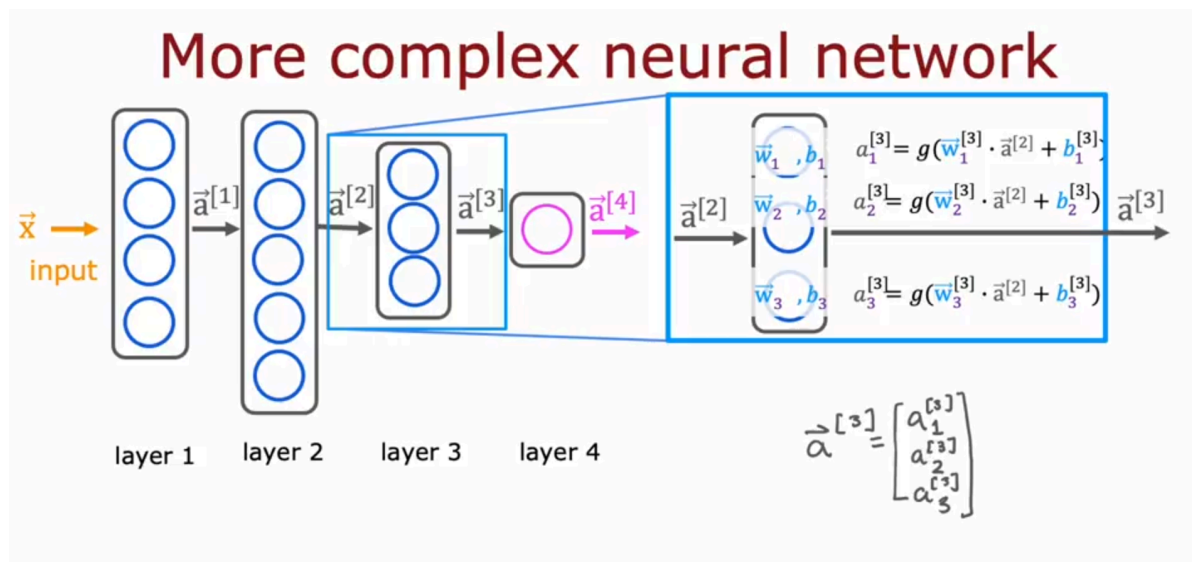
1. Layers in the Network:

- **Input Layer (Layer 0):**
 - Takes in raw input features (e.g., price, shipping cost).
 - Not counted when determining the number of layers in a neural network.
- **Hidden Layers (Layers 1, 2, and 3):**

- Perform intermediate computations.
- Extract features from the input data.
- **Output Layer (Layer 4):**
 - Produces the final prediction.

2. Counting Layers:

- By convention, the total number of layers includes all hidden layers and the output layer but excludes the input layer.
- Example: A neural network with 3 hidden layers and 1 output layer is referred to as a "4-layer network."



Zooming into Layer 3 (Final Hidden Layer)

Inputs and Outputs:

- Input to Layer 3: Activation vector from Layer 2, denoted as $a[2]$.
- Output of Layer 3: Activation vector $a[3]$, which becomes the input to Layer 4.

Neuron Computations in Layer 3:

1. Each neuron in Layer 3 performs logistic regression-like computations: $a_j[3] = g(w_j[3] \cdot a[2] + b_j[3])$

$$a_j[3] = g(w_j[3] \cdot a[2] + b_j[3])$$

Where:

- $w_j[3]$: Weight vector for neuron jj in Layer 3.
 - $b_j[3]$: Bias term for neuron jj in Layer 3.
 - $g(z) = \frac{1}{1 + e^{-z}}$: Sigmoid activation function.
2. Example for three neurons in Layer 3:
 - Neuron 1: $a_1[3] = g(w_1[3] \cdot a[2] + b_1[3])$
 $a_1[3] = g(w_1[3] \cdot a[2] + b_1[3])$
 - Neuron 2: $a_2[3] = g(w_2[3] \cdot a[2] + b_2[3])$
 $a_2[3] = g(w_2[3] \cdot a[2] + b_2[3])$
 - Neuron 3: $a_3[3] = g(w_3[3] \cdot a[2] + b_3[3])$
 $a_3[3] = g(w_3[3] \cdot a[2] + b_3[3])$

Output of Layer 3:

- The output is an activation vector comprising activations from all neurons in Layer 3: $a[3] = [a_1[3], a_2[3], a_3[3]]$
 $a[3] = [a_1[3], a_2[3], a_3[3]]$

General Notation for Neural Networks

1. Activation Values:

- $a_j[l]$: Activation value of neuron jj in layer l .
- Example: $a_2[3]$ is the activation of the second neuron in Layer 3.

2. Weights and Biases:

- $w_j[l]$: Weight vector for neuron jj in layer l .
- $b_j[l]$: Bias term for neuron jj in layer l .

- Example: $w_1[2], b_1[2]$ are parameters for the first neuron in Layer 2.

3. Input to Each Layer:

- Input to layer l is the activation vector from layer $l-1$, denoted as $a[l-1]$.

4. General Equation for Any Neuron: $z_j[l] = w_j[l] \cdot a[l-1] + b_j[l]$ $a_j[l] = g(z_j[l])$

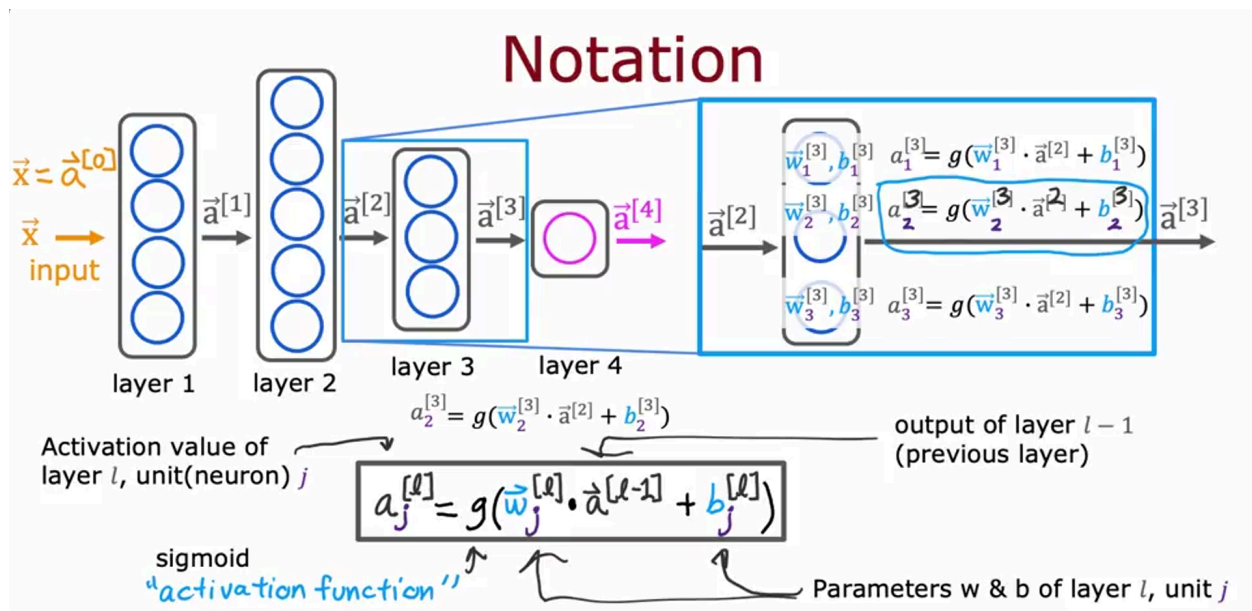
For neuron j in layer l :

$$z_j[l] = w_j[l] \cdot a[l-1] + b_j[l]$$

$$a_j[l] = g(z_j[l])$$

5. Input Features as Activations:

- The input feature vector (x) is treated as the activations of the input layer (a).
- This ensures consistent notation across all layers.



Activation Function

1. Definition:

- The activation function ($g(z)$) determines whether a neuron should be activated or not.
- It transforms the weighted sum of inputs (z) into an output value (a).

2. Sigmoid Function:

- Formula: $g(z) = \frac{1}{1 + e^{-z}}$
- Output range: $[0, 1]$.
- Commonly used for binary classification tasks.

Inference Algorithm for Neural Networks

Forward Propagation:

- To make predictions, activations are computed sequentially from one layer to the next using forward propagation.
- Steps:
 1. Compute activations for each neuron in Layer 1 using input features (X):

$$z_j[1] = w_j[1] \cdot a + b_j[1]$$

$$a_j[1] = g(z_j[1])$$

$$z_j[1] = w_j[1] \cdot a + b_j[1]$$

$$a_j[1] = g(z_j[1])$$
 2. Use activations from Layer 1 ($a[1]$) as input to compute activations for neurons in Layer 2.
Repeat similar computations for subsequent layers.
 3. Continue this process until reaching the output layer.

Final Prediction:

- The output of the final layer provides either:
 - A probability (e.g., likelihood of being a top seller).
 - A binary classification (e.g., threshold at 0.5).

Key Takeaways

- Neural networks consist of multiple layers where each neuron computes an activation value based on inputs from the previous layer.
- Notation helps track parameters (w, b, w', b') and activations (a, z, a', z') across layers.
- The sigmoid function is commonly used as an activation function but can be replaced with other functions depending on requirements.
- Forward propagation enables neural networks to compute predictions by passing data through successive layers.

Next Steps

In future sessions, alternative activation functions will be introduced (beyond sigmoid). These functions can improve performance and address specific challenges in training deep neural networks. Additionally, more advanced architectures and inference algorithms will be explored to handle larger and more complex datasets effectively.

Overview of the Inference Algorithm

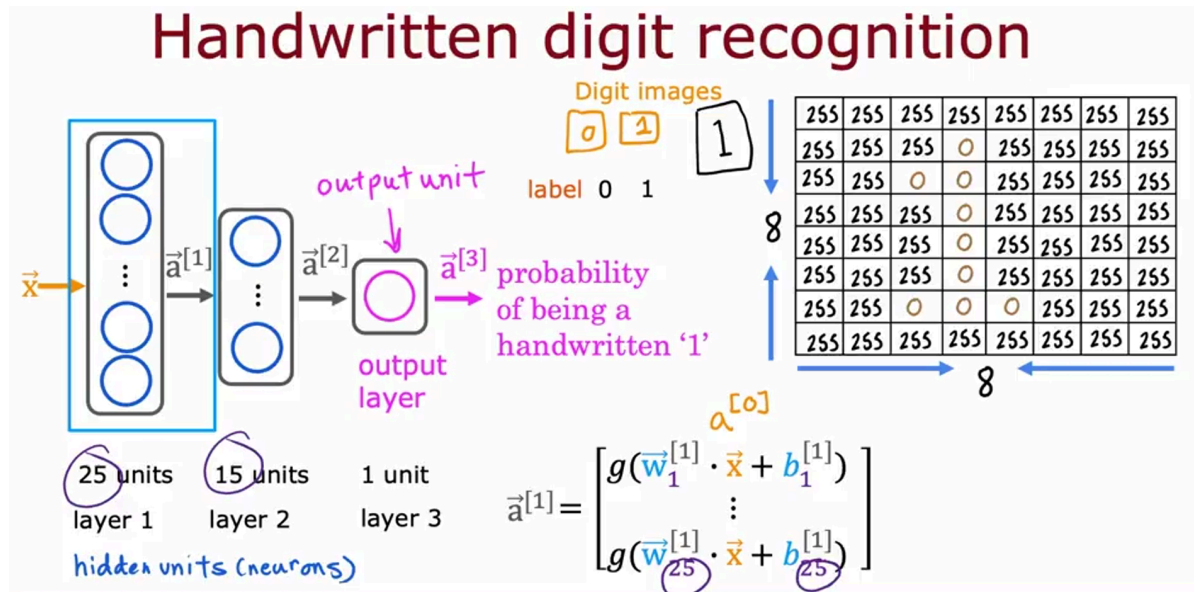
- The algorithm discussed is called **forward propagation**, used to let a neural network make inferences (predictions) from input data.
- The motivating example is handwritten digit recognition, specifically distinguishing between the digits zero and one (a binary classification problem).

Input and Problem Setup

- **Input Data:**
 - A handwritten digit is represented as an 8×8 image.
 - The image is a matrix with 64 pixel intensity values.
 - Each pixel value ranges from 0 (black) to 255 (bright white), with intermediate values representing shades of gray.

- **Task:**

- Classify the input image as either digit 0 or digit 1.
- Optionally, after computing the network's output probability, a threshold can be applied to decide between a binary classification label ("0" or "1").



Neural Network Architecture

- **Overall Structure:**

- The network consists of multiple layers.
- By convention, the input layer is referred to as **Layer 0** and is not counted when specifying the network depth.

- **Layer Breakdown:**

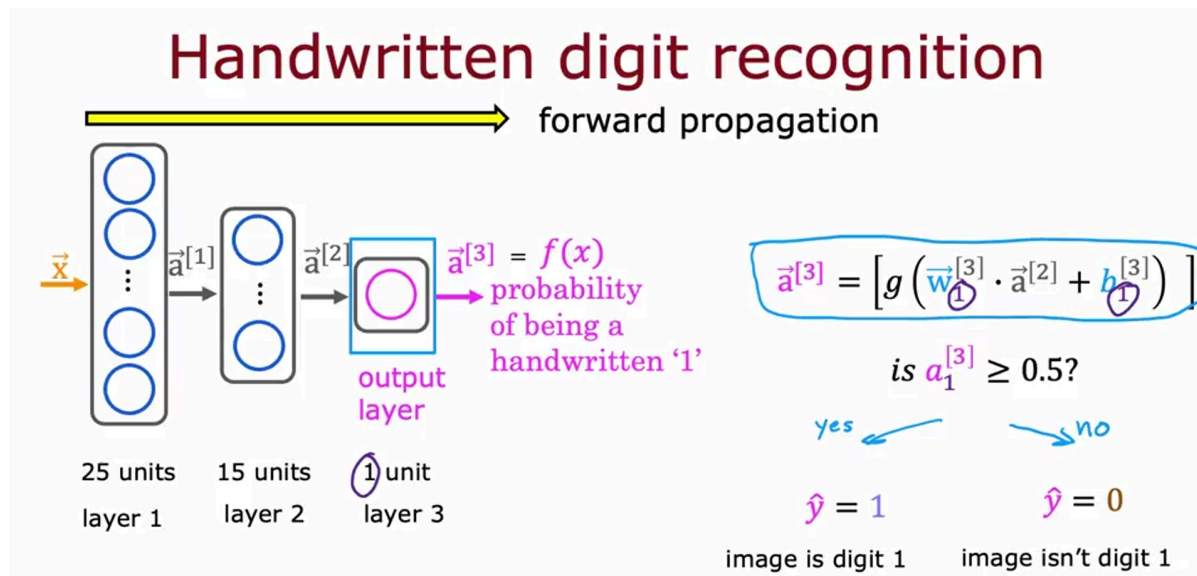
- **Layer 0 (Input Layer):**

- Contains all 64 input features from the 8×8 image.

- **Hidden Layers:**

- **First Hidden Layer (Layer 1):**

- Contains 25 neurons (units).
- Computes the activation vector $a[1]a[1]$ of dimension 25.
- **Second Hidden Layer (Layer 2):**
 - Contains 15 neurons (units).
 - Computes the activation vector $a[2]a[2]$ of dimension 15.
- **Output Layer (Layer 3):**
 - A single neuron providing the network's output $a[3]a[3]$.
 - The scalar output represents the probability that the input is digit 1.



- **Architectural Pattern:**

- The network uses a common design: starting with a larger number of hidden neurons that reduces as you progress closer to the output layer.
- This "bottleneck" design is typical in many applications.

Step-by-Step Computation (Forward Propagation Process)

1. From Input $\vec{x}$ (or $a[1]a[1]$) to First Hidden Layer ($a[1]a[1]$):

- **Operation:**

- Each neuron in Layer 1 performs a weighted sum of the input features plus a bias, then applies an activation function.

- Mathematically: $zj[1] = wj[1] \cdot a + bj[1]$ $aj[1] = g(zj[1])$

$$zj[1] = wj[1] \cdot a + bj[1]$$

$$aj[1] = g(zj[1])$$

- j ranges over the 25 neurons.

- **Result:**

- The output is a 25-dimensional vector $a[1]$ representing intermediate features computed from the input image.

2. From First Hidden Layer $a[1]$ to Second Hidden Layer ($a[2]$):

- **Operation:**

- The second layer computes its activations from $a[1]$ using its own set of weights and biases.

- For each of the 15 neurons in Layer 2: $zj[2] = wj[2] \cdot a[1] + bj[2]$ $aj[2] = g(zj[2])$

$$zj[2] = wj[2] \cdot a[1] + bj[2]$$

$$aj[2] = g(zj[2])$$

- The activation function g (shown earlier as the sigmoid in other examples, though here just generically noted) is applied element-wise.

- **Result:**

- The output is a 15-dimensional vector $a[2]$ that summarizes higher-level features from the first hidden layer.

3. From Second Hidden Layer $a[2]$ to Output Layer ($a[3]$):

- **Operation:**

- The output layer contains a single neuron that computes its output as: $z[3] = w[3] \cdot a[2] + b[3]$ $a[3] = g(z[3])$

$$z[3] = w[3] \cdot a[2] + b[3]$$

$$a[3] = g(z[3])$$

- Here, the dot product uses the weights of the output neuron and $a[2]a[2]$ serves as its input.

- **Result:**

- The final output $a[3]a[3]$ is a scalar probability.
- This value signifies the likelihood that the input image is of the digit 1.

4. Optional Thresholding for Binary Classification:

- The scalar output $a[3]a[3]$ can be compared to a threshold (for example, 0.5 or another value) to yield a binary classification:
 - If $a[3]a[3]$ exceeds the threshold, predict digit 1.
 - Otherwise, predict digit 0.
- Also noted: The network's function can be symbolically written as $f(x)=a[3]f(x)=a[3]$, analogous to how we denote outputs for linear or logistic regression.

General Observations and Notational Clarifications

- **Forward Propagation:**

- It entails computing the activations sequentially from the input layer through all hidden layers to the output.
- The computation is performed in a forward (left-to-right) manner.
- This process "propagates" the inputs through the network to arrive at a prediction.

- **Activation Function gg:**

- It is the non-linear function applied at each neuron after computing the weighted input sum.
- In many cases, the sigmoid function is used for simple binary classification, though alternatives (e.g., ReLU, tanh) might be used in different contexts.
- The role of the activation function is crucial to introduce non-linearity and allows the network to learn complex patterns.

- **Notation Recap:**

- a : Input feature vector (or activation of the input layer).
- $a[l]$: Output (activations) of layer l .
- $w[l]$ and $b[l]$: Weights and biases for neurons of layer l .
- $z[j]$: The linear combination (weighted sum plus bias) for neuron j in layer l .
- $g(\cdot)$: Activation function applied to $z[j]$.

- **Design Philosophy:**

- The structure, where layers gradually reduce in size (from 25 to 15 units before the single output), is a common architectural choice.
- It helps in capturing and refining features as the network advances toward final classification.

- **Practical Implication:**

- Once the forward propagation algorithm is understood and the math for computing activations is clear, you can perform inference with a pre-trained neural network by simply "feeding forward" new input data.
- This prepares the ground for practical implementations using frameworks such as TensorFlow. In the next video, the instructor plans to demonstrate an implementation of these concepts using TensorFlow.

- **Contrast with Backpropagation:**

- Forward propagation is strictly for computing the output/prediction from input data.
- It is distinct from backpropagation, where the network parameters (weights and biases) are updated using the gradients of the loss function.
- In this session, backpropagation is only mentioned briefly as the counterpart for learning, with further details promised in the following week's lesson.

These detailed notes comprehensively capture every point and explanation provided in the transcript. The forward propagation algorithm is explained step-

by-step, covering the transformation from the 8×8 input image through two hidden layers to the final output probability, along with associated notations and practical implications for inference using a neural network.

LAB 1: Neurons and Layers

Packages

Tensorflow and Keras

Tensorflow is a machine learning package developed by Google. In 2019, Google integrated Keras into Tensorflow and released Tensorflow 2.0. Keras is a framework developed independently by François Chollet that creates a simple, layer-centric interface to Tensorflow. This course will be using the Keras interface.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras import Sequential
from tensorflow.keras.losses import MeanSquaredError, BinaryCrossentropy
from tensorflow.keras.activations import sigmoid
from lab_utils_common import dlc
from lab_neurons_utils import plt_prob_1d, sigmoidnp, plt_linear, plt_logistic
plt.style.use('./deeplearning.mplstyle')
import logging
logging.getLogger("tensorflow").setLevel(logging.ERROR)
tf.autograph.set_verbosity(0)
```


Neuron without activation - Regression/Linear Model

DataSet

We'll use an example from Course 1, linear regression on house prices.

```
X_train = np.array([[1.0], [2.0]], dtype=np.float32)      #(size in 1000 square feet)
Y_train = np.array([[300.0], [500.0]], dtype=np.float32)  #(price in 1000s of dollars)

fig, ax = plt.subplots(1,1)
ax.scatter(X_train, Y_train, marker='x', c='r', label="Data Points")
ax.legend( fontsize='xx-large')
ax.set_ylabel('Price (in 1000s of dollars)', fontsize='xx-large')
ax.set_xlabel('Size (1000 sqft)', fontsize='xx-large')
plt.show()
```

Regression/Linear Model

The function implemented by a neuron with no activation is the same as in Course 1, linear regression: $f_{\{\mathbf{w}\},b}(x^{(i)}) = \mathbf{w} \cdot \mathbf{x}^{(i)} + b$

We can define a layer with one neuron or unit and compare it to the familiar linear regression function.

```
linear_layer = tf.keras.layers.Dense(units=1, activation = 'linear', )
```

```
#examine the weights
linear_layer.get_weights()
```

```
# There are no weights as the weights are not yet instantiated. Let's try the model
a1 = linear_layer(X_train[0].reshape(1,1))
print(a1)
```

```
# The result is a tensor (another name for an array) with a shape of (1,1) or one element
# Now let's look at the weights and bias. These weights are randomly initialized to zero
w, b = linear_layer.get_weights()
print(f"w = {w}, b={b}")
```

```
"""A linear regression model (1) with a single input feature will have a single weight
and a single bias. The weights are initialized to random values so let's set them to some known values"""
```

```
set_w = np.array([[200]])
set_b = np.array([100])

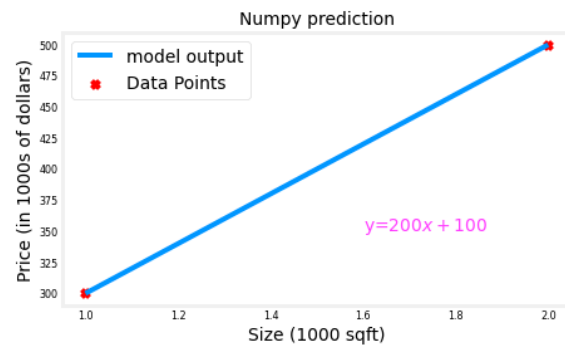
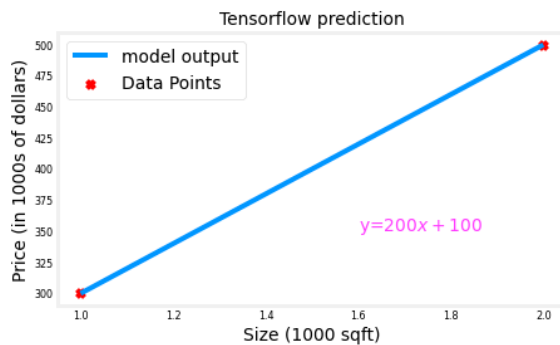
# set_weights takes a list of numpy arrays
linear_layer.set_weights([set_w, set_b])
print(linear_layer.get_weights())
```

equation vs output

```
a1 = linear_layer(X_train[0].reshape(1,1))
print(a1)
alin = np.dot(set_w, X_train[0].reshape(1,1)) + set_b
print(alin)
```

```
prediction_tf = linear_layer(X_train)
prediction_np = np.dot( X_train, set_w) + set_b
```

```
plt_linear(X_train, Y_train, prediction_tf, prediction_np)
```



Neuron with Sigmoid activation ¶

The function implemented by a neuron/unit with a sigmoid activation is the same as in Course 1, logistic regression: $f_{\mathbf{w}, \mathbf{b}}(\mathbf{x}^{(i)}) = g(\mathbf{w}^T \mathbf{x}^{(i)} + b)$ where $g(x) = \text{sigmoid}(x)$

Let's set $\mathbf{w}$ and $\mathbf{b}$ to some known values and check the model.

DataSet

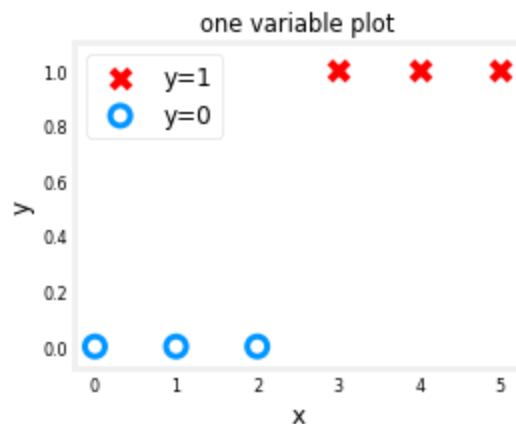
We'll use an example from Course 1, logistic regression.

```
X_train = np.array([0., 1, 2, 3, 4, 5], dtype=np.float32).reshape(-1,1) # 2-D Matrix
Y_train = np.array([0, 0, 0, 1, 1, 1], dtype=np.float32).reshape(-1,1) # 2-D Matrix
```

```
pos = Y_train == 1
neg = Y_train == 0
```

```
fig,ax = plt.subplots(1,1,figsize=(4,3))
ax.scatter(X_train[pos], Y_train[pos], marker='x', s=80, c = 'red', label="y=1")
ax.scatter(X_train[neg], Y_train[neg], marker='o', s=100, label="y=0", facecolors:
          edgecolors=dlc["dlblue"],lw=3)
```

```
ax.set_ylim(-0.08,1.1)
ax.set_ylabel('y', fontsize=12)
ax.set_xlabel('x', fontsize=12)
ax.set_title('one variable plot')
ax.legend(fontsize=12)
plt.show()
```



Logistic Neuron

We can implement a 'logistic neuron' by adding a sigmoid activation. The function of the neuron is then described by (2) above.

This section will create a Tensorflow Model that contains our logistic layer to demonstrate an alternate method of creating models. Tensorflow is most often used to create multi-layer models. The [Sequential](#) model is a convenient means of constructing these models.

```

model = Sequential(
    [
        tf.keras.layers.Dense(1, input_dim=1, activation = 'sigmoid', name='L1')
    ]
)

```

```

model.summary()
#shows the layers and number of parameters in the model. There is only one layer

```

```

logistic_layer = model.get_layer('L1')
w,b = logistic_layer.get_weights()
print(w,b)
print(w.shape,b.shape)

```

#setting weights and bias to known values

```

set_w = np.array([[2]])
set_b = np.array([-4.5])
# set_weights takes a list of numpy arrays
logistic_layer.set_weights([set_w, set_b])
print(logistic_layer.get_weights())

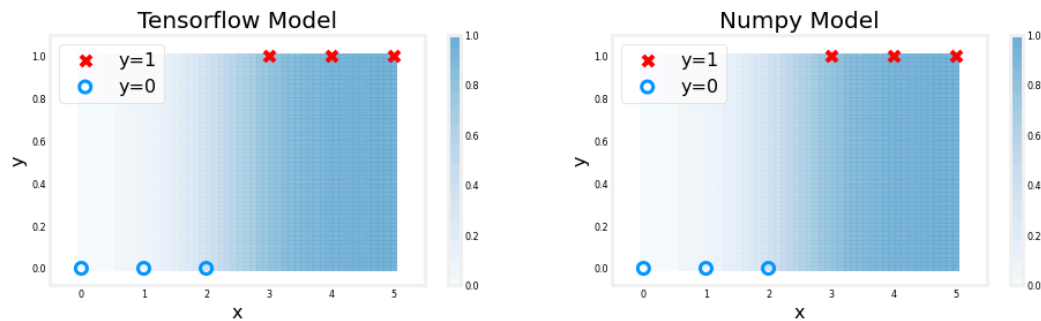
```

```

a1 = model.predict(X_train[0].reshape(1,1))
print(a1)
alog = sigmoidnp(np.dot(set_w,X_train[0].reshape(1,1)) + set_b)
print(alog)

```

```
plt_logistic(X_train, Y_train, model, set_w, set_b, pos, neg)
```



The shading above reflects the output of the sigmoid which varies from 0 to 1.

Done

Inference in code

Introduction and Context

- TensorFlow is one of the leading frameworks for implementing deep learning algorithms.
- Even though PyTorch is also popular, this specialization focuses on TensorFlow.
- The video explains how to write inferencing code using TensorFlow, i.e. how to perform forward propagation to get predictions.

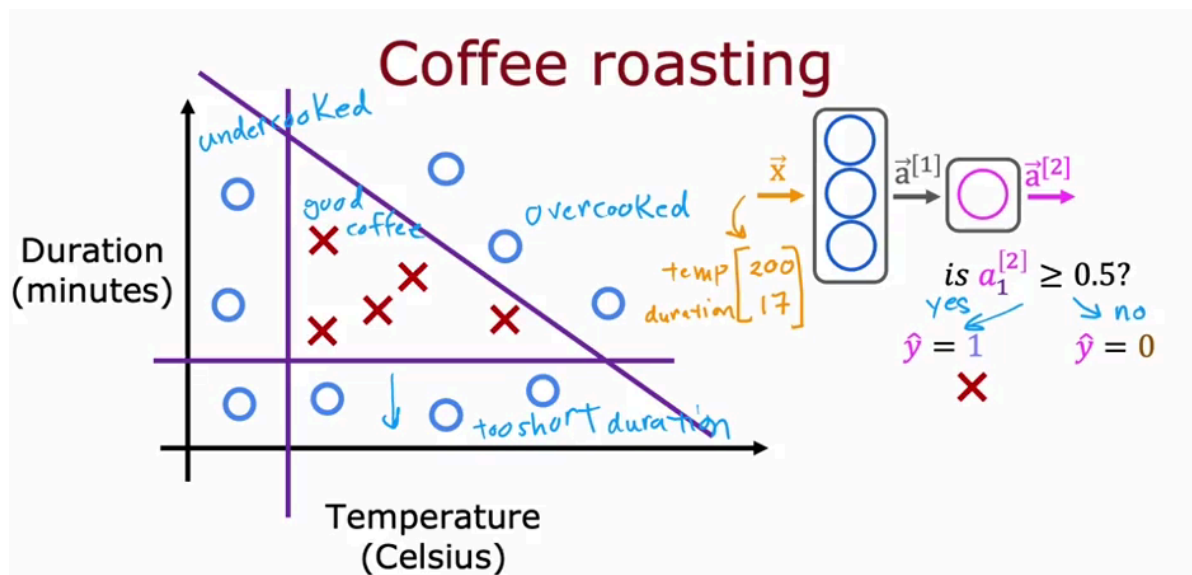
General Inference Concept in Neural Networks

- Neural networks use the same algorithm across many applications.

- The inference process involves passing an input (feature vector) through one or more dense (fully connected) layers.
- The result (output activation) is obtained via forward propagation from input to the final output layer.
- Optionally, the final output can be thresholded (e.g., at 0.5) to yield a binary prediction.

Example 1: Coffee Roasting Inference

- **Motivation:**
 - The example illustrates how machine learning can optimize coffee roasting.
 - Personal example: The speaker roasts Colombian coffee beans at home.
 - The goal is to determine whether a given roasting process will yield good-tasting coffee.
- **Dataset Characteristics:**
 - Two controllable parameters:
 - **Temperature:** The heat level during roasting.
 - **Duration:** The length of time the beans are roasted.
 - Labels in the dataset:
 - **Positive label ($y = 1$):** Indicates good (well-roasted) coffee.
 - **Negative label:** Indicates bad (under-roasted or over-roasted) coffee.
 - Practical observation:
 - Roasting at too low a temperature or with too short a duration leads to undercooked beans.
 - Roasting at too high a temperature or for too long leads to burnt, overcooked beans.
 - Only data points falling within a certain optimal “triangle” in the temperature–duration space result in good coffee.

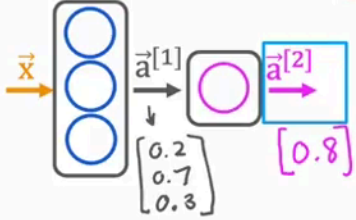


- **Inference Process for Coffee Roasting:**
 - **Step 1: Define Input Features**
 - Example: An array representing [200 (degrees Celsius), 17 (minutes)].
 - These two numbers form the feature vector $\mathbf{x}$.
 - **Step 2: Create the First Hidden Layer (Layer 1)**
 - Implemented as a dense layer with **3 units (neurons)**.
 - Uses the **sigmoid activation function**.
 - Compute:
 - $\mathbf{a}_1 = \text{Layer 1}(\mathbf{x})$
 - Example outcome: $\mathbf{a}_1$ might be [0.2, 0.7, 0.3] (three activation values).
 - **Step 3: Create the Second Hidden Layer (Layer 2)**
 - Implemented as a dense layer with **1 unit**.
 - Again uses the **sigmoid activation function**.
 - Compute:
 - $\mathbf{a}_2 = \text{Layer 2}(\mathbf{a}_1)$
 - $\mathbf{a}_2$ is a scalar value representing the network's output probability.

- **Step 4: Optional Label Thresholding**

- Compare the scalar a_2 with a threshold (typically 0.5):
 - If $a_2 \geq 0.5$, set prediction $\hat{y} = 1$ (good coffee).
 - Otherwise, $\hat{y} = 0$ (bad coffee).

Build the model using TensorFlow



is $a_1^{[2]} \geq 0.5$?

yes $\hat{y} = 1$ ✗

no $\hat{y} = 0$ ○

```

x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)

layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
  
```

```

if a2 >= 0.5:
    yhat = 1
else:
    yhat = 0
  
```

- **Additional Note:**

- Some details (like loading the TensorFlow library and the neural network's parameters—weights and biases) are not covered in the video and will be addressed later in the lab.

Example 2: Handwritten Digit Classification Inference

- **Context and Data Structure:**

- In this example, the task is to distinguish handwritten digits (typically, a binary classification between two digits or extended to multiclass later).

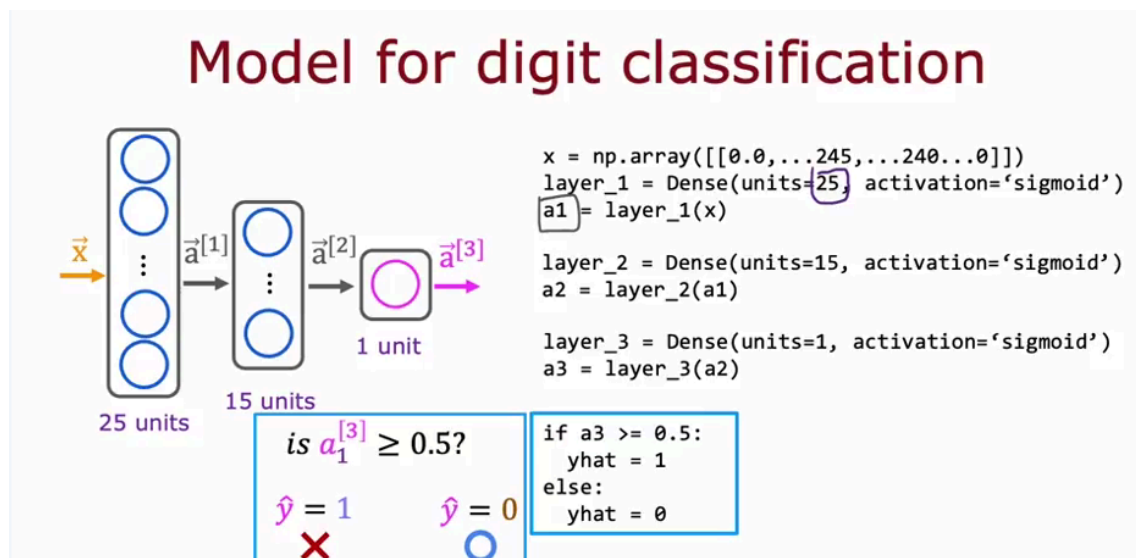
- **Input x :**

- Represented as a NumPy array of pixel intensity values.
- Each pixel value corresponds to a shade (with values typically normalized).

- **Inference Process for Digit Classification:**

- **Step 1: Build the First Hidden Layer (Layer 1)**

- Use a **dense layer** with **25 units**.
- Employ the **sigmoid activation function**.
- Compute:
 - $a_1 = \text{Layer 1}(x)$
- **Step 2: Build the Second Hidden Layer (Layer 2)**
 - Again, set up another dense layer (details such as number of units follow similar principles).
 - Compute:
 - $a_2 = \text{Layer 2}(a_1)$
- **Step 3: Build the Third (Final) Dense Layer (Layer 3)**
 - This is the output layer.
 - Compute:
 - $a_3 = \text{Layer 3}(a_2)$
- **Step 4: Optional Thresholding for Final Prediction**
 - The scalar output a_3 can be compared to a threshold (e.g., 0.5) to determine the final binary prediction ($\hat{y}$).



- **Key Syntax and Implementation Points:**

- The approach essentially follows forward propagation: passing the input through successive dense layers.
- The structure and dimensions of NumPy arrays are important, as TensorFlow expects data in particular formats.
- **Final Practical Note:**
 - The video concludes by noting that additional details (such as data handling and TensorFlow's treatment of NumPy arrays) will be covered in the next session.

Summary and Takeaways:

- TensorFlow is widely used for deep learning inference.
- The inference procedure (forward propagation) involves:
 - Creating dense layers with specified units and activation functions (here, sigmoid).
 - Sequentially computing the activation for each layer starting with the input vector x .
 - Optionally applying thresholding to produce a binary classification.
- Two examples were used:
 - **Coffee Roasting:** Inference from temperature and duration input to decide on coffee quality.
 - **Handwritten Digits:** Inference from pixel intensity values for digit classification.
- Some implementation details (loading libraries, handling model parameters, and array formatting) are to be explored further in the lab and subsequent videos.

These notes capture the complete details and steps discussed in the transcript regarding implementing neural network inference using TensorFlow.

Data in Tensorflow

This session focuses on how data is represented in **NumPy** and **TensorFlow**, highlighting the differences and conventions used in each framework. It also explains how matrices, vectors, and tensors are used to implement neural networks efficiently.

Introduction

- **NumPy:**
 - A standard Python library for linear algebra, widely used for data manipulation.
 - Represents data as arrays (1D, 2D, etc.).

Feature vectors

temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
...	...	...

```
x = np.array([[200.0, 17.0]])
```

←

[[200.0, 17.0]]

why?

Note about numpy arrays

2 rows

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

3 columns

2 x 3 matrix

```
x = np.array([[1, 2, 3], [4, 5, 6]])
```

[[1, 2, 3],
[4, 5, 6]]

2D array

2 x 3

4 rows

$$\begin{bmatrix} 0.1 & 0.2 \\ -3 & -4 \\ -5 & -6 \\ 7 & 8 \end{bmatrix}$$

2 columns

4 x 2 matrix

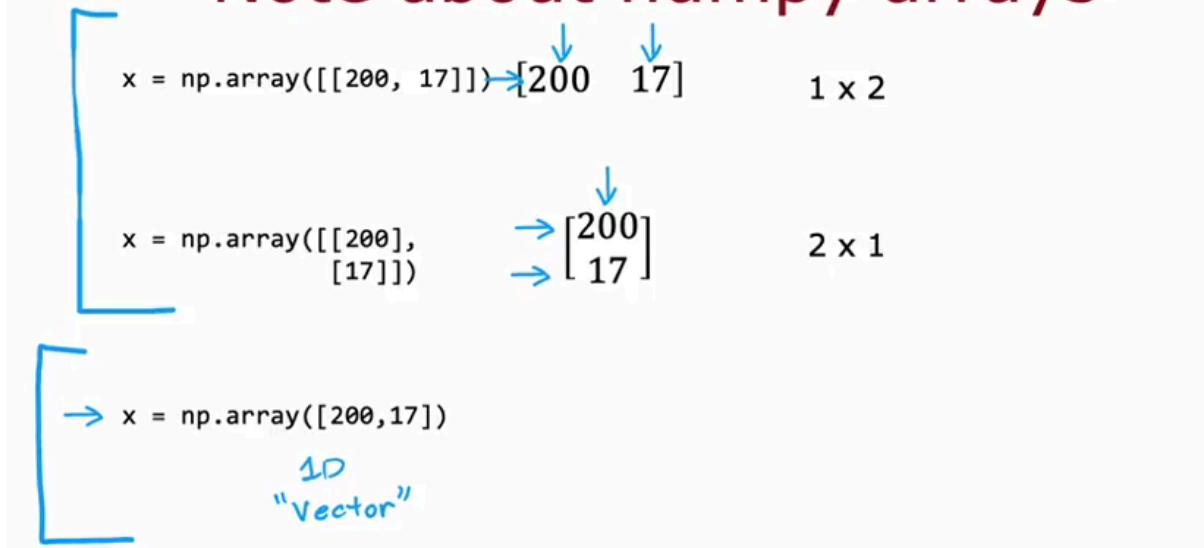
```
x = np.array([[0.1, 0.2], [-3.0, -4.0], [-0.5, -0.6], [7.0, 8.0]])
```

[[0.1, 0.2],
[-3.0, -4.0],
[-0.5, -0.6],
[7.0, 8.0]]

4 x 2

1 x

Note about numpy arrays



- **TensorFlow:**
 - A deep learning framework designed for efficient computation on large datasets.
 - Represents data as **tensors**, which are generalizations of matrices.
- **Key Difference:**
 - TensorFlow uses matrices (2D arrays) instead of 1D arrays for data representation to optimize computational efficiency.
 - This difference stems from the historical development of the two libraries.

Matrix Representation in NumPy

Matrix Dimensions

- A matrix is a **2D array of numbers** with rows and columns.
- Convention: The dimensions of a matrix are written as **rows × columns**.

Examples

1. 2×3 Matrix:

- Example:
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$
- Code Representation in NumPy:

```
pythonx = np.array([[1, 2, 3], [4, 5, 6]])
```
- Explanation:
 - `[[1, ]]` groups rows together.
 - The result is a matrix with:
 - **2 rows:** `[1]` and ````.
 - **3 columns:** Each row has three elements.

2. 4×2 Matrix:

- Example:
$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}$$
- Code Representation in NumPy:

```
pythonx = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])
```
- Explanation:
 - Four rows: `[1]`, `,`, ````.
 - Two columns.

Special Cases of Matrices

Row Vector ($1 \times n$ Matrix):

- Example:[200,17]
[200, 17][200, \;17]
- Code Representation in NumPy:

```
pythonx = np.array([[200, 17]])
```
- Explanation:
 - A single row with two columns.
 - Dimensions: **1×2 matrix**.

Column Vector ($n \times 1$ Matrix):

- Example:[20017]
[20017]\begin{bmatrix}200 \\ 17\end{bmatrix}
- Code Representation in NumPy:

```
pythonx = np.array([[200], [17]])
```
- Explanation:
 - Two rows with one column.
 - Dimensions: **2×1 matrix**.

1D Array (No Rows or Columns):

- Example:

```
pythonx = np.array([200, 17])
```
- Explanation:
 - A simple list of numbers without explicit rows or columns.
 - This is a **linear array**, not a matrix.

Data Representation in TensorFlow

Why TensorFlow Uses Matrices Instead of Arrays

- TensorFlow was designed to handle large datasets efficiently.
- Representing data as matrices (even when one dimension is 1) allows TensorFlow to optimize internal computations.

Tensors

- In TensorFlow, data is represented as tensors.
- A tensor is a generalization of matrices but can be thought of as a matrix for most practical purposes in this course.

Examples

1. When input features are represented as a matrix:

```
pythonx = tf.constant([[200.0, 17.0]]) # Shape: (1, 2)
```

- This creates a tensor with shape (1 × 2) (one row and two columns).
- TensorFlow treats this as a floating-point number (float32), which uses **32 bits of memory per value**.

2. Converting between TensorFlow tensors and NumPy arrays:

```
python # Convert TensorFlow tensor to NumPy array  
numpy_array = tensor.numpy()
```

Forward Propagation Example

Step-by-Step Computation

First Hidden Layer (Layer 1)

- Input feature vector xxx:

[200,;17](a(a(a1 \times n\$\$ matrix).

- Layer setup in TensorFlow:
-


```
pythonlayer_1 = tf.keras.layers.Dense(units=3, activation='sigmoid')
```

- Compute activations for Layer 1:

```
pythona_1 = layer_1(x)
```

- a_1 has shape $1 \times n$ where n is the number of units (here $n=3$).
- Example output:

$[0.2, 0.7, 0.3]$

Second Hidden Layer (Layer 2)

- Layer setup in TensorFlow:

```
pythonlayer_2 = tf.keras.layers.Dense(units=1, activation='sigmoid')
```

- Compute activations for Layer 2:

```
pythona_2 = layer_2(a_1)
```

- a_2 has shape $1 \times n$ where $n=1$.
- Example output:

$[0.8]$

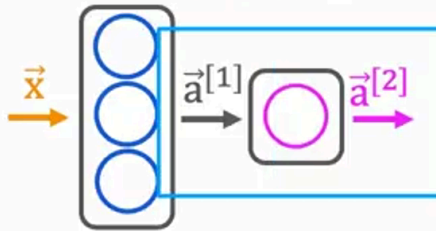
Final Output

- Thresholding at 0.5:

```
pythony_hat = int(a_2 >= 0.5)
```

- If $a_2 \geq 0.5 \rightarrow \text{Predict } y_{\text{hat}} = \text{Positive Class } (y=1)$.
- Else $\rightarrow \text{Predict } y_{\text{hat}} = \text{Negative Class } (y=0)$.

Activation vector



```
layer_2 = Dense(units=1, activation='sigmoid')  
a2 = layer_2(a1)
```

```
tf.Tensor([[0.8]], shape=(1, 1), dtype=float32)
```

```
a2.numpy()
```

```
array([[0.8]], dtype=float32)
```

Key Observations

Differences Between NumPy and TensorFlow Representations

- NumPy often uses simpler representations like linear arrays for small datasets.
- TensorFlow prefers matrices (even for single-row or single-column data) for computational efficiency.

Conversions Between Formats

- Convert TensorFlow tensors to NumPy arrays using `.numpy()` method.
- Convert NumPy arrays to TensorFlow tensors using `tf.convert_to_tensor()`.

Summary

Key Takeaways

- Matrices are fundamental to representing data in both NumPy and TensorFlow.

- Special cases include row vectors ($1 \times n$) and column vectors ($n \times 1$).
- TensorFlow uses tensors (generalized matrices) to optimize computations on large datasets.
- Forward propagation involves passing input through layers to compute activations sequentially.
- Conversions between NumPy arrays and TensorFlow tensors allow seamless integration between the two libraries.

Practical Implications

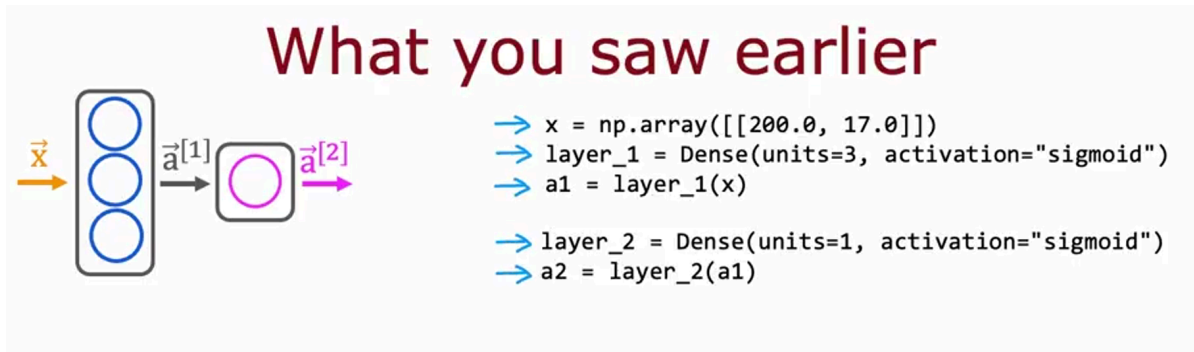
- Understanding these conventions ensures correct implementation of neural networks in TensorFlow.
- While conventions may seem complex initially, practice with code will clarify these concepts further.

This concludes the detailed notes on how data is represented and processed in NumPy and TensorFlow. The next session will focus on building neural networks using these principles.

Building a Neural Network

Traditional Forward Propagation Approach

- **Manual Process (Previously Learned):**
 - **Step 1:** Initialize input data (x).
 - **Step 2:** Create Layer 1 and compute its activations (a_1).
 - **Step 3:** Create Layer 2 and compute its activations (a_2).
 - The process involves explicitly passing the input through each layer one at a time.

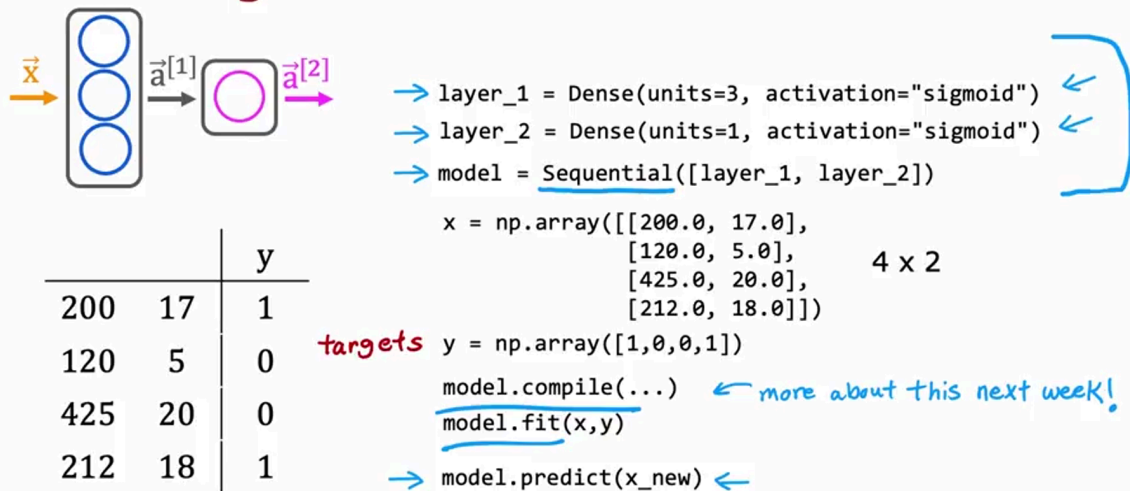


The Sequential Function in TensorFlow

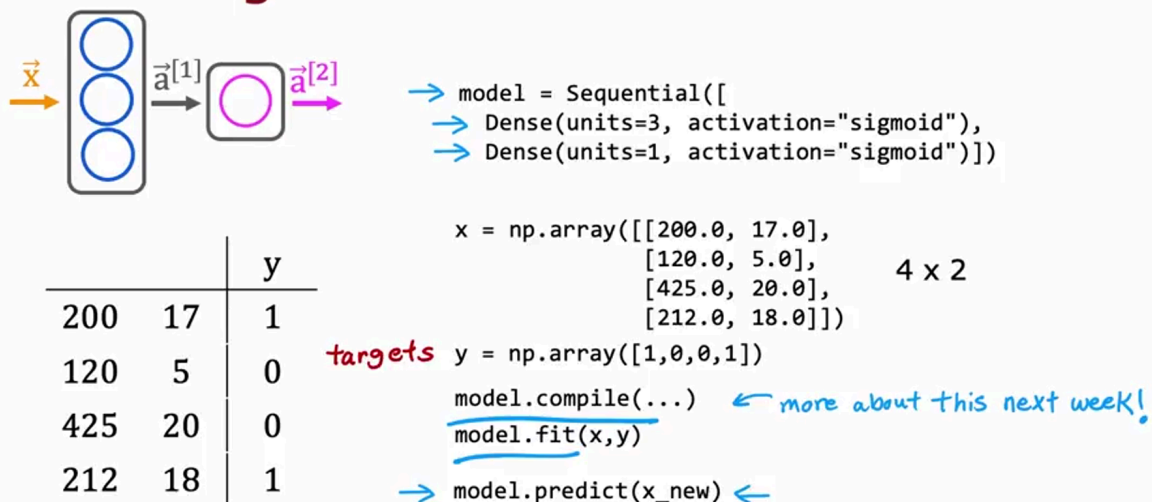
- **New Approach for Building Neural Networks:**
 - Instead of manually chaining each layer, you can instruct TensorFlow to "string together" layers using the **sequential function**.
 - This function automatically composes multiple layers into a single neural network model.
- **How It Works:**
 - **Step 1:** Define each layer (e.g., Layer 1, Layer 2).
 - **Step 2:** Use the `Sequential` function to combine these layers.
 - For example:

```
pythonmodel = tf.keras.Sequential([
    tf.keras.layers.Dense(units=3, activation='sigmoid'),
    tf.keras.layers.Dense(units=1, activation='sigmoid')
])
```
 - TensorFlow then handles passing the input from one layer to the next (forward propagation) automatically.

Building a neural network architecture



Building a neural network architecture



Example 1: Coffee Roasting Classification

- Dataset for the Coffee Example:

- Input (x):

- Stored as a NumPy array representing a 4x2 matrix (four training examples, each with two features: temperature and duration).

- Target Labels (y):

- A 1-D array of length 4 (e.g., ``), where:
 - 1 indicates good-tasting coffee.
 - 0 indicates bad coffee.
- **Training the Neural Network:**
 - Once the model is created using the sequential function, you:
 - **Compile the Model:**
 - Using `model.compile()` with appropriate parameters (details to be covered next week).
 - **Fit the Model:**
 - Call `model.fit(X, Y)` to train the neural network on the training data.
- **Performing Inference:**
 - To get predictions on new data (`X_new`):
 - Call `model.predict(X_new)`, which carries out forward propagation through the network and outputs the prediction (the activation from the last layer).

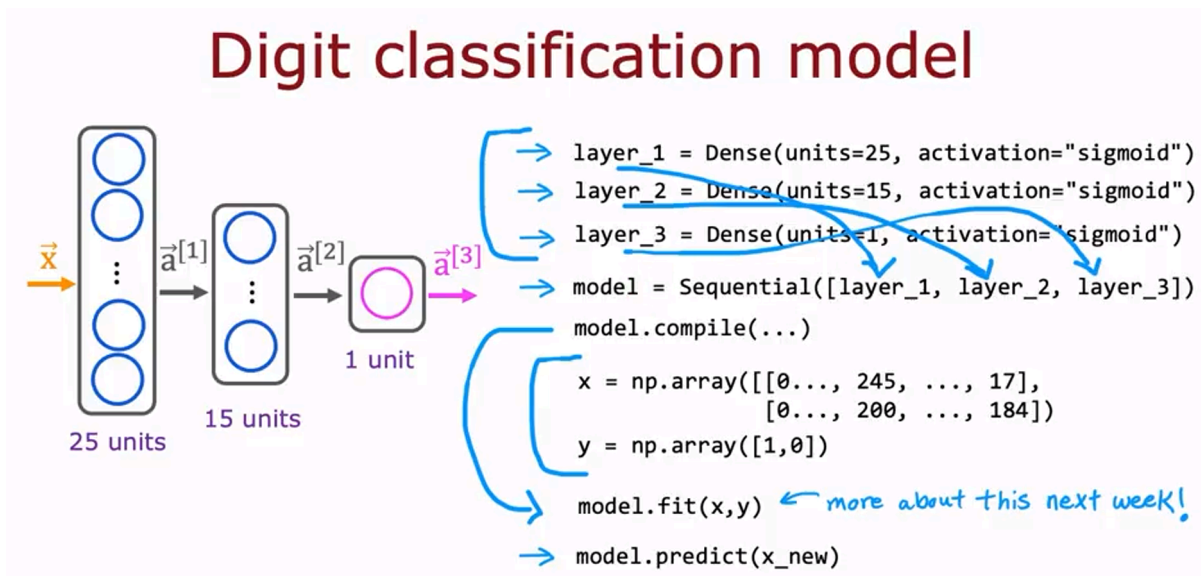
Example 2: Handwritten Digit Classification

- **Scenario:**
 - The input `X` consists of pixel intensity values for digits.
- **Model Construction Using Sequential API:**
 - **Old Method:**
 - Previously, you would manually create each layer (e.g., compute `a1`, then `a2`, then `a3`).
 - **New Method with Sequential:**
 - Define layers (e.g., Layer 1 with 25 units, Layer 2 with a certain number, Layer 3 as the output layer) and pass them directly into the `Sequential` function.
 - Example compact code:

```
pythonmodel = tf.keras.Sequential([
    tf.keras.layers.Dense(units=25, activation='sigmoid'),
    tf.keras.layers.Dense(units=<some_number>, activation='sigmoid'),
    tf.keras.layers.Dense(units=1, activation='sigmoid')
])
```

o Training and Inference:

- Like before, store input data in an array (matrix format), compile the model, fit the model with `model.fit(X, Y)`, and perform predictions using `model.predict(X_new)`.



Simplified Coding Conventions

• More Compact Code:

- Instead of assigning each layer to a separate variable (e.g., `layer_1`, `layer_2`), it is common to pass the layers directly into the `Sequential` constructor.
- This results in minimal but powerful code—only a few lines are needed to build, compile, and use a neural network for both training and inference.

• Understanding Behind the Scenes:

- While the sequential API simplifies coding, it's important to know what happens internally:
 - Layers are automatically connected.

- Forward propagation is managed by TensorFlow.
- The instructor emphasizes understanding the underlying process even while using these high-level APIs.

Final Thoughts and Next Steps

- **Takeaway from the Specialization:**

- Leverage cutting-edge libraries like TensorFlow to build efficient neural networks with minimal code.
- Do not rely solely on libraries—understand what the code is doing to troubleshoot and innovate if needed.

- **Preview of Future Content:**

- In an upcoming video, you will learn to implement forward propagation from scratch in Python.
- This exercise helps deepen your understanding of how algorithms work beyond using high-level APIs.

- **Summary:**

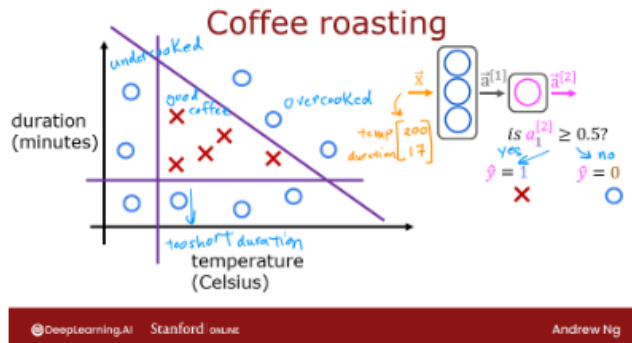
- The sequential API is an efficient way to build a neural network by chaining layers together.
- Training and inference are simplified to a few lines of code.
- A solid grasp of the internals aids you in debugging and refining machine learning models in real-world applications.

These notes thoroughly encapsulate the transcript, covering every step and detail of building, training, and inferencing with a neural network in TensorFlow using the sequential API.

LAB 2: Simple Neural Network

Optional Lab - Simple Neural Network

In this lab we will build a small neural network using Tensorflow.



```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('./deeplearning.mplstyle')
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from lab_utils_common import dlc
from lab_coffee_utils import load_coffee_data, plt_roast, plt_prob, plt_layer, plt_ne
import logging
logging.getLogger("tensorflow").setLevel(logging.ERROR)
tf.autograph.set_verbosity(0)
```

Loading data

```
X,Y = load_coffee_data();
print(X.shape, Y.shape)
```

plot

```
plt_roast(X,Y)
```

Normalize Data

Fitting the weights to the data (back-propagation, covered in next week's lectures) will proceed more quickly if the data is normalized. This is the same procedure you used in Course 1 where features in the data are each normalized to have a similar range. The procedure below uses a Keras [normalization layer](#). It has the following steps:

- create a "Normalization Layer". Note, as applied here, this is not a layer in your model.
- 'adapt' the data. This learns the mean and variance of the data set and saves the values internally.
- normalize the data.

It is important to apply normalization to any future data that utilizes the learned model.

** Normalization layer

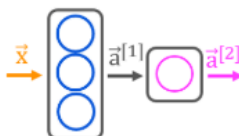
```
print(f"Temperature Max, Min pre normalization: {np.max(X[:,0]):0.2f}, {np.min(X[:,0]):0.2f}")
print(f"Duration Max, Min pre normalization: {np.max(X[:,1]):0.2f}, {np.min(X[:,1]):0.2f}")
norm_l = tf.keras.layers.Normalization(axis=-1)
norm_l.adapt(X) # learns mean, variance
Xn = norm_l(X)
print(f"Temperature Max, Min post normalization: {np.max(Xn[:,0]):0.2f}, {np.min(Xn[:,0]):0.2f}")
print(f"Duration Max, Min post normalization: {np.max(Xn[:,1]):0.2f}, {np.min(Xn[:,1]):0.2f}")
```

Tile/copy our data to increase the training set size and reduce the number of training epochs.

```
Xt = np.tile(Xn,(1000,1))
Yt= np.tile(Y,(1000,1))
print(Xt.shape, Yt.shape)
```

Tensorflow Model

Model



Let's build the "Coffee Roasting Network" described in lecture. There are two layers with sigmoid activations as shown below:

```
tf.random.set_seed(1234) # applied to achieve consistent results
model = Sequential(
    [
        tf.keras.Input(shape=(2,)),
        Dense(3, activation='sigmoid', name = 'layer1'),
        Dense(1, activation='sigmoid', name = 'layer2')
    ]
)
```

Note 1: The `tf.keras.Input(shape=(2,))`, specifies the expected shape of the input. This allows Tensorflow to size the weights and bias parameters at this point. This is useful when exploring Tensorflow models. This statement can be omitted in practice and Tensorflow will size the network parameters when the input data is specified in the `model.fit` statement.

Note 2: Including the sigmoid activation in the final layer is not considered best practice. It would instead be accounted for in the loss which improves numerical stability. This will be described in more detail in a later lab.

The `model.summary()` provides a description of the network:

```
model.summary()
```

The parameter counts shown in the summary correspond to the number of elements in the weight and bias arrays as shown below.

```
L1_num_params = 2 * 3 + 3 # W1 parameters + b1 parameters
L2_num_params = 3 * 1 + 1 # W2 parameters + b2 parameters
print("L1 params = ", L1_num_params, ", L2 params = ", L2_num_params )
```

Let's examine the weights and biases Tensorflow has instantiated. The weights W should be of size (number of features in input, number of units in the layer) while the bias b size should match the number of units in the layer:

- In the first layer with 3 units, we expect W to have a size of (2,3) and \$b\$ should have 3 elements.
- In the second layer with 1 unit, we expect W to have a size of (3,1) and \$b\$ should have 1 element.

```
W1, b1 = model.get_layer("layer1").get_weights()
W2, b2 = model.get_layer("layer2").get_weights()
print(f"W1{W1.shape}:\n", W1, f"\nb1{b1.shape}:", b1)
print(f"W2{W2.shape}:\n", W2, f"\nb2{b2.shape}:", b2)
```

The following statements will be described in detail in Week2. For now:

- The `model.compile` statement defines a loss function and specifies a compile optimization.
- The `model.fit` statement runs gradient descent and fits the weights to the data.

```
model.compile(
    loss = tf.keras.losses.BinaryCrossentropy(),
    optimizer = tf.keras.optimizers.Adam(learning_rate=0.01),
)

model.fit(
    Xt,Yt,
    epochs=10,
)
```

Epochs and batches

In the `fit` statement above, the number of `epochs` was set to 10. This specifies that the entire data set should be applied during training 10 times. During training, you see output describing the progress of training that looks like this:

```
Epoch 1/10  
6250/6250 [=====] - 6s 910us/step - loss: 0.1782
```

The first line, `Epoch 1/10`, describes which epoch the model is currently running. For efficiency, the training data set is broken into 'batches'. The default size of a batch in Tensorflow is 32. There are 200000 examples in our expanded data set or 6250 batches. The notation on the 2nd line `6250/6250 [=====]` is describing which batch has been executed.

Updated Weights

After fitting, the weights have been updated:

```
W1, b1 = model.get_layer("layer1").get_weights()  
W2, b2 = model.get_layer("layer2").get_weights()  
print("W1:\n", W1, "\nb1:", b1)  
print("W2:\n", W2, "\nb2:", b2)
```

You can see that the values are different from what you printed before calling `model.fit()`. With these, the model should be able to discern what is a good or bad coffee roast.

For the purpose of the next discussion, instead of using the weights you got right away, you will first set some weights we saved from a previous training run. This is so that this notebook remains robust to changes in Tensorflow over time. Different training runs can produce somewhat different results and the following discussion applies when the model has the weights you will load below.

Feel free to re-run the notebook later with the cell below commented out to see if there is any difference. If you got a low loss after the training above (e.g. 0.002), then you will most likely get the same results.

```
# After finishing the lab later, you can re-run all  
# cells except this one to see if your trained model  
# gets the same results.
```

```
# Set weights from a previous run.
```

```
W1 = np.array([  
    [-8.94, 0.29, 12.89],  
    [-0.17, -7.34, 10.79]])  
b1 = np.array([-9.87, -9.28, 1.01])  
W2 = np.array([
```

```

[-31.38],
[-27.86],
[-32.79]])
b2 = np.array([15.54])

# Replace the weights from your trained model with
# the values above.
model.get_layer("layer1").set_weights([W1,b1])
model.get_layer("layer2").set_weights([W2,b2])

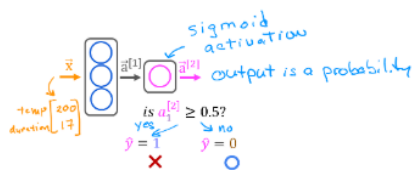
```

```

# Check if the weights are successfully replaced
W1, b1 = model.get_layer("layer1").get_weights()
W2, b2 = model.get_layer("layer2").get_weights()
print("W1:\n", W1, "\nb1:", b1)
print("W2:\n", W2, "\nb2:", b2)

```

Predictions



Once you have a trained model, you can then use it to make predictions. Recall that the output of our model is a probability. In this case, the probability of a good roast. To make a decision, one must apply the probability to a threshold. In this case, we will use 0.5

Let's start by creating input data. The model is expecting one or more examples where examples are in the rows of matrix. In this case, we have two features so the matrix will be (m,2) where m is the number of examples. Recall, we have normalized the input features so we must normalize our test data as well. To make a prediction, you apply the `predict` method.

```

X_test = np.array([
    [200,13.9], # positive example
    [200,17]]) # negative example
X_testn = norm_l(X_test)

```

```
predictions = model.predict(X_testn)
print("predictions = \n", predictions)
```

Threshold: to convert probability into a decision

```
yhat = np.zeros_like(predictions)
for i in range(len(predictions)):
    if predictions[i] >= 0.5:
        yhat[i] = 1
    else:
        yhat[i] = 0
print(f"decisions = \n{yhat}")
```

```
yhat = (predictions >= 0.5).astype(int)
print(f"decisions = \n{yhat}")
```

Layer Functions

Let's examine the functions of the units to determine their role in the coffee roasting decision. We will plot the output of each node for all values of the inputs (duration,temp). Each unit is a logistic function whose output can range from zero to one. The shading in the graph represents the output value.

Note: In labs we typically number things starting at zero while the lectures may start with 1

```
plt_layer(X,Y.reshape(-1,),W1,b1,norm_l)
```

The shading shows that each unit is responsible for a different "bad roast" region. unit 0 has larger values when the temperature is too low. unit 1 has larger values when the duration is too short and unit 2 has larger values for bad combinations of time/temp. It is worth noting that the network learned these functions on its own through the process of gradient descent. They are very much the same sort of functions a person might choose to make the same decisions.

The function plot of the final layer is a bit more difficult to visualize. It's inputs are the output of the first layer. We know that the first layer uses sigmoids so their output range is between zero and one. We can create a 3-D plot that calculates the output for all possible combinations of the three inputs. This is shown below. Above, high output values correspond to 'bad roast' area's. Below, the maximum output is in area's where the three inputs are small values corresponding to 'good roast' area's.

```
plt_output_unit(W2,b2)
```

The final graph shows the whole network in action.

The left graph is the raw output of the final layer represented by the blue shading. This is overlaid on the training data represented by the X's and O's.

The right graph is the output of the network after a decision threshold. The X's and O's here correspond to decisions made by the network.

The following takes a moment to run

```
netf= lambda x : model.predict(norm_l(x))  
plt_network(X,Y,netf)
```

Done

Forward Propagation in Python

Overview and Context

- The session reviews previous lessons on:
 - Building a single layer in TensorFlow.
 - Performing forward propagation through one layer.
 - Representing data in TensorFlow.
- The focus now is on combining everything learned to build an entire neural network in TensorFlow.
- Two main approaches are highlighted:
 - The explicit, step-by-step forward propagation (manually passing data layer by layer).
 - A simpler, higher-level method using TensorFlow's `Sequential` API to automatically string layers together.

Traditional Forward Propagation in TensorFlow (Explicit Method)

- **Prior Process Recap:**
 - Initialize the input data (x).
 - Create the first hidden layer.
 - Compute its activations (a_1) by passing x through the layer.
 - Create the second hidden layer.
 - Compute its activations (a_2) by passing a_1 through the layer.
- This approach involves manually passing inputs or activation outputs from one layer to the next.

The Sequential API: A Simpler Approach

- **Concept:**

- Instead of manually managing the flow of data through each layer, TensorFlow provides the `Sequential` function.
- With `Sequential`, you declare the layers, and TensorFlow automatically strings them together.

- **How It Works:**

- You define each layer (for example, a dense layer with a specified number of units and an activation function).
- You pass these layers as a list to `tf.keras.Sequential()`.
- TensorFlow then handles both forward propagation and later, during training, backpropagation.

- **Example Workflow:**

- **For a Coffee Roasting Model:**

- Training data:

- `X`: A 4×2 matrix (four training examples, each with two features — temperature and duration).
- `Y`: A 1-D array of target labels (e.g., `11`), where 1 indicates good coffee.

- Model creation using the sequential method:

- Define Layer 1 (e.g., dense with 3 units and sigmoid activation).
- Define Layer 2 (e.g., dense with 1 unit and sigmoid activation).
- Chain them in one compact `Sequential` call.

- **Training:**

- Compile the model (using `model.compile(...)` with chosen optimizer and loss function).
- Fit the model on the training data (`model.fit(X, Y)`).

- **Inference:**
 - Use `model.predict(X_new)` to perform forward propagation automatically on new input data.
- **For Handwritten Digit Classification:**
 - Similar approach with sequential chaining:
 - Typically more layers (e.g., Layer 1, Layer 2, Layer 3) are defined.
 - The same principles apply to compile, fit, and predict.
- **Coding Style:**
 - It is common practice to directly pass layers into the `Sequential` constructor instead of assigning them to individual variables.
 - This results in more concise code while still allowing the user to benefit from underlying forward propagation and training methods.

Emphasis on Understanding the Underlying Process

- **Why It Matters:**
 - Although using TensorFlow's high-level API results in just a few lines of code, the underlying process of forward propagation remains key.
 - Understanding the underlying mechanics is essential for:
 - Debugging and troubleshooting.
 - Extending or modifying the model architecture.
 - Gaining the insight needed to implement such concepts from scratch if a new framework is developed.
- **Preview of Next Steps:**
 - The instructor explains that the next video will address implementing forward propagation from scratch in Python.
 - This exercise will reveal the detailed computations and help build intuition about what libraries like TensorFlow and PyTorch are doing under the hood.

Implementing Forward Propagation from Scratch Using Python (Detailed Example)

- **Motivation:**

- To understand internal workings and gain a deeper intuition about neural network computations.
- This is especially useful if you ever decide to experiment beyond high-level APIs.

- **Example Model: Coffee Roasting Model**

- **Representation:**

- Input feature vector x represents parameters like temperature and duration.
- Use 1D arrays in Python (using single square bracket notation) to represent vectors and parameters.

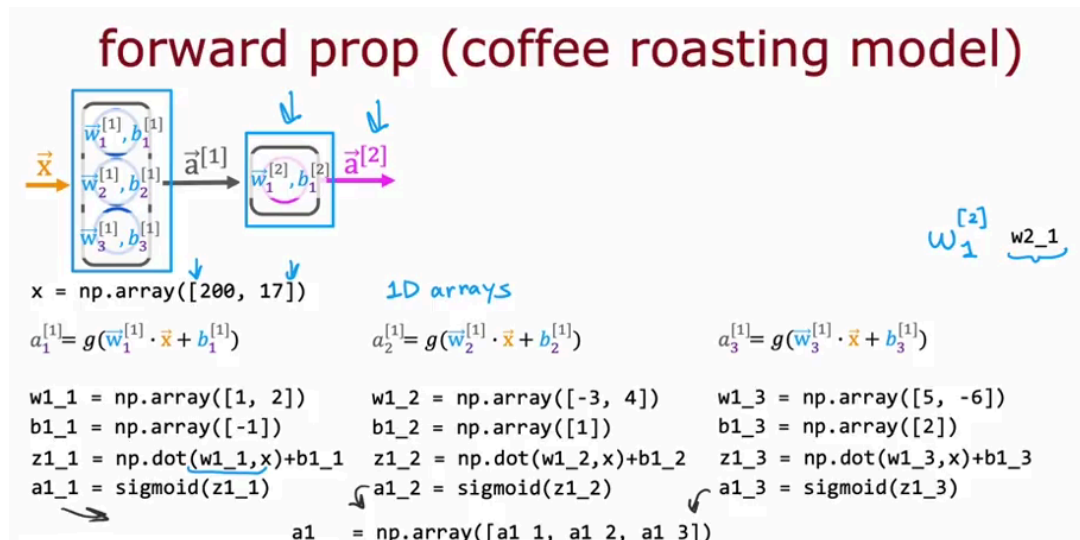
- **First Layer Calculation:**

- **For Each Neuron in the First Layer (e.g., three neurons):**

- Compute the first neuron's activation (a_{11}):
 - Calculate the weighted sum:
 - Dot product of w_{11} (first neuron's weight) and x , add bias b_{11} .
 - Apply the sigmoid activation function:
 - $a_{11} = \text{sigmoid}(w_{11} \cdot x + b_{11})$.
- Similarly, compute a_{12} :
 - Using weights w_{12} , bias b_{12} .
- Similarly, compute a_{13} :
 - Using weights w_{13} , bias b_{13} .

- **Aggregate Activations:**

- Group a_{11} , a_{12} , and a_{13} into an array to form a_1 , the output of the first layer.



■ Second Layer Calculation:

• For the Output Neuron:

- Use parameters w_{21} and b_{21} .
- Compute the dot product between w_{21} and the first layer output a_1 .
- Add the bias b_{21} and apply the sigmoid function to get a_{21} .
- The result, a_{21} , is the output of the neural network (final inference).

$$a_1^{[2]} = g(\bar{w}_1^{[2]} \cdot \bar{a}^{[1]} + b_1^{[2]})$$

→ $w2_1 = \text{np.array}([-7, 8, 9])$
→ $b2_1 = \text{np.array}([3])$
→ $z2_1 = \text{np.dot}(w2_1, a1) + b2_1$
 $a2_1 = \text{sigmoid}(z2_1)$

- **Note on Notation:**

- The transcript uses notation such as:
 - w_{21} to denote the weight for the first neuron in the second layer.
 - a_{11} , a_{12} , etc., denote the activation values for corresponding neurons.

- **Outcome:**

- This step-by-step manual implementation demonstrates the elementary operations performed during forward propagation.
- It lays the groundwork for later generalizing the implementation so the code is not hard-coded for every neuron.

Conclusion and Next Topics

- **Summary:**

- The session transitions from using TensorFlow's high-level APIs to a deeper understanding of forward propagation by implementing it from scratch.
- Both the sequential API approach and manual implementation offer valuable insights:
 - **Sequential API:** Simplifies development with compact code.
 - **Manual Implementation:** Enhances understanding of model internals and internal computational steps.

- **Looking Ahead:**

- The next video will focus on further simplifying and generalizing the manual implementation for more complex neural networks.
- This deep dive is intended to build intuition that will empower you to modify, troubleshoot, or even innovate new approaches in neural network frameworks.

These detailed notes capture every major point and explanation from the transcript, including the use of the Sequential API in TensorFlow, examples using

coffee roasting and handwritten digit classification models, and the foundational process of implementing forward propagation manually from scratch.

Implementation

Implementing Forward Propagation in Python

1. Introduction

- Previously, forward propagation was implemented by **hardcoding** computations for each neuron.
 - This session covers a **generalized implementation** of forward propagation using Python functions.
 - Understanding this will help in interpreting similar code in **practice labs**.
-

2. Implementing a Dense Layer

What is a Dense Layer?

- A **dense layer** is a fully connected layer where each neuron is connected to every neuron in the previous layer.
- It requires **inputs**, **weights (W)**, and **biases (b)** to compute **activations** for the next layer.

Defining the Dense Function

```
def dense(a, W, b):
```

- **Inputs:**
 - **a**: Activation from the previous layer.

- `W`: Weight matrix for the current layer.
- `b`: Bias vector for the current layer.
- **Output:**
 - The computed activation values for the next layer.

Stacking Weights and Biases into Matrices

- **Weights (`W`):**
 - If a layer has 3 neurons, weights are stacked into a **2×3 matrix**.
 - Example:

```
W = [[w_1,1, w_1,2, w_1,3],
     [w_2,1, w_2,2, w_2,3]]
```

- **Biases (`b`):**
 - Stored as a **1D array**: `[b_1,1, b_1,2, b_1,3]`

3. Forward Propagation Code Breakdown

```
units = W.shape[1] # Number of units (neurons) in the layer
a = np.zeros(units) # Initialize activations to zero
```

- `W.shape[1]`: Extracts **number of neurons** from the weight matrix.
- `np.zeros(units)`: Creates an array of **zeros** for storing activations.

Computing Activations

```
for j in range(units):
    w = W[:, j] # Extract j-th column (weights for neuron j)
    z = np.dot(w, a) + b[j] # Compute weighted sum + bias
    a[j] = sigmoid(z) # Apply activation function
```

- Extracts weights for each neuron using `W[:, j]`.

- Computes **linear transformation** ($z = Wx + b$).
- Applies the **sigmoid activation function**.

Returning Activations

```
return a
```

- Outputs activation values for the current layer.

4. Implementing Forward Propagation with Multiple Layers

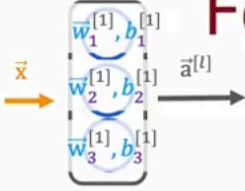
- Forward propagation consists of **sequentially computing activations** across multiple layers.
- Using the `dense()` function, a neural network with **4 layers** can be implemented as:

```
a1 = dense(x, W1, b1)
a2 = dense(a1, W2, b2)
a3 = dense(a2, W3, b3)
a4 = dense(a3, W4, b4)
f_x = a4 # Final output of the network
```

Notational Convention

- **Uppercase `w`**: Indicates matrices (weight matrix).
- **Lowercase `b`**: Indicates vectors (bias vector).

Forward prop in NumPy



$\vec{x}$ → $\vec{a}^{[l]}$

$\vec{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ $\vec{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$ $\vec{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$

$W = \text{np.array}(\begin{bmatrix} 1 & -3 & 5 \\ 2 & 4 & -6 \end{bmatrix})$ *2 by 3*

$b_1^{[l]} = -1$ $b_2^{[l]} = 1$ $b_3^{[l]} = 2$

$b = \text{np.array}([-1, 1, 2])$

$\vec{a}^{[0]} = \vec{x}$

$a_in = \text{np.array}([-2, 4])$

```

def dense(a_in, W, b):
    3 units = W.shape[1]    [0,0,0]
    a_out = np.zeros(units)
    for j in range(units): 0,1,2
        w = W[:,j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out
    
```

capital W refers to a matrix

```

def sequential(x):
    a1 = dense(x, W1, b1)
    a2 = dense(a1, W2, b2)
    a3 = dense(a2, W3, b3)
    a4 = dense(a3, W4, b4)
    f_x = a4
    return f_x
    
```

Note: g() is defined outside of dense().
(see optional lab for details)

5. Why Understanding Forward Propagation Matters

- Even when using libraries like **TensorFlow**, understanding forward propagation:
 - Helps **debug errors** in deep learning models.
 - Improves **optimization and performance tuning**.
 - Makes it easier to **interpret and modify models**.

Debugging Machine Learning Models

- **Most ML models don't work on the first try.**
- Debugging skills are **crucial** for fixing issues related to:
 - Slow performance.
 - Unexpected results.
 - Incorrect parameter updates.

6. Next Topic: Neural Networks and Artificial General Intelligence (AGI)

- Discussion on whether **neural networks contribute to AGI**.

- Understanding the relationship between **modern AI** and **human-level intelligence**.

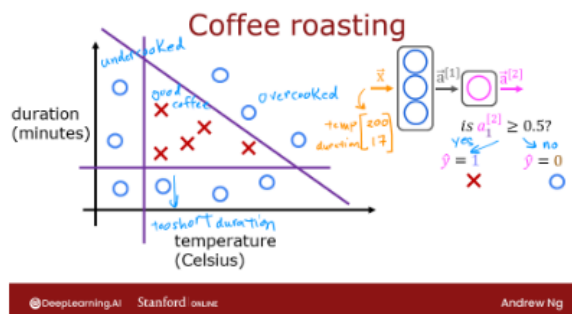
Summary

- **Dense layers** take activations, weights, and biases to compute new activations.
- **Forward propagation** is implemented by **stacking dense layers** sequentially.
- Understanding forward propagation **improves debugging and optimization** in ML.
- **Next class** explores the connection between **neural networks and AGI**.

LAB 3: Simple Neural Network

Optional Lab - Simple Neural Network

In this lab, we will build a small neural network using Numpy. It will be the same "coffee roasting" network you implemented in Tensorflow.



```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('./deeplearning.mplstyle')
import tensorflow as tf
```

```

from lab_utils_common import dlc, sigmoid
from lab_coffee_utils import load_coffee_data, plt_roast, plt_prob, plt_layer, plt_ne
import logging
logging.getLogger("tensorflow").setLevel(logging.ERROR)
tf.autograph.set_verbosity(0)

```

load data

```

X,Y = load_coffee_data();
print(X.shape, Y.shape)

```

Let's plot the coffee roasting data below. The two features are Temperature in Celsius and Duration in minutes. [Coffee Roasting at Home](#) suggests that the duration is best kept between 12 and 15 minutes while the temp should be between 175 and 260 degrees Celsius. Of course, as the temperature rises, the duration should shrink.

```

plt_roast(X,Y)

```

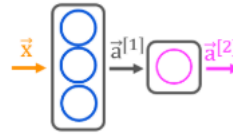
Normalize

```

print(f"Temperature Max, Min pre normalization: {np.max(X[:,0]):0.2f}, {np.min(X[:,0]):0.2f}")
print(f"Duration   Max, Min pre normalization: {np.max(X[:,1]):0.2f}, {np.min(X[:,1]):0.2f}")
norm_l = tf.keras.layers.Normalization(axis=-1)
norm_l.adapt(X) # learns mean, variance
Xn = norm_l(X)
print(f"Temperature Max, Min post normalization: {np.max(Xn[:,0]):0.2f}, {np.min(Xn[:,0]):0.2f}")
print(f"Duration   Max, Min post normalization: {np.max(Xn[:,1]):0.2f}, {np.min(Xn[:,1]):0.2f}")

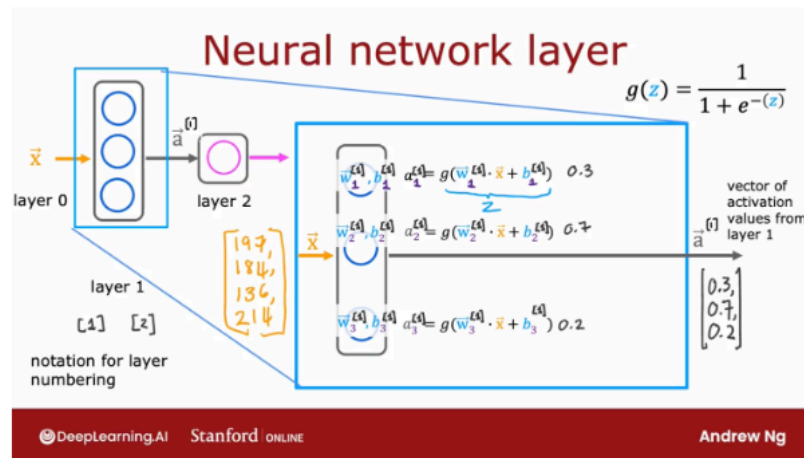
```

Numpy Model (Forward Prop in NumPy)



Let's build the "Coffee Roasting Network" described in lecture. There are two layers with sigmoid activations.

As described in lecture, it is possible to build your own dense layer using NumPy. This can then be utilized to build a multi-layer neural network.



In the first optional lab, you constructed a neuron in NumPy and in TensorFlow and noted their similarity. A layer simply contains multiple neurons/units. As described in lecture, one can utilize a for loop to visit each unit (j) in the layer and perform the dot product of the weights for that unit ($w[:, j]$) and sum the bias for the unit ($b[j]$) to form z . An activation function $g(z)$ can then be applied to that result. Let's try that below to build a "dense layer" subroutine.

First, you will define the activation function $g()$. You will use the `sigmoid()` function which is already implemented for you in the `lab_utils_common.py` file outside this notebook.

```
# Define the activation function
g = sigmoid
```

Next, you will define the `my_dense()` function which computes the activations of a dense layer.

```
def my_dense(a_in, W, b):
    """
    Computes dense layer
    Args:
```

```

a_in (ndarray (n, )) : Data, 1 example
W   (ndarray (n,j)) : Weight matrix, n features per unit, j units
b   (ndarray (j, )) : bias vector, j units
Returns
a_out (ndarray (j,)) : j units|
"""
units = W.shape[1]
a_out = np.zeros(units)
for j in range(units):
    w = W[:,j]
    z = np.dot(w, a_in) + b[j]
    a_out[j] = g(z)
return(a_out)

```

Note: You can also implement the function above to accept `g` as an additional parameter (e.g. `my_dense(a_in, W, b, g)`). In this notebook though, you will only use one type of activation function (i.e. sigmoid) so it's okay to make it constant and define it outside the function. That's what you did in the code above and it makes the function calls in the next code cells simpler. Just keep in mind that passing it as a parameter is also an acceptable implementation. You will see that in this week's assignment.

The following cell builds a two-layer neural network utilizing the `my_dense` subroutine above.

```

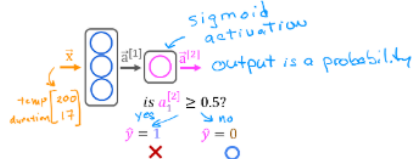
def my_sequential(x, W1, b1, W2, b2):
    a1 = my_dense(x, W1, b1)
    a2 = my_dense(a1, W2, b2)
    return(a2)

```

We can copy trained weights and biases from the previous lab in Tensorflow.

```
W1_tmp = np.array( [[-8.93, 0.29, 12.9 ], [-0.1, -7.32, 10.81]] )
b1_tmp = np.array( [-9.82, -9.28, 0.96] )
W2_tmp = np.array( [[-31.18], [-27.59], [-32.56]] )
b2_tmp = np.array( [15.41] )
```

Predictions



Once you have a trained model, you can then use it to make predictions. Recall that the output of our model is a probability. In this case, the probability of a good roast. To make a decision, one must apply the probability to a threshold. In this case, we will use 0.5

Let's start by writing a routine similar to Tensorflow's `model.predict()`. This will take a matrix X with all m examples in the rows and make a prediction by running the model.

```
def my_predict(X, W1, b1, W2, b2):
    m = X.shape[0]
    p = np.zeros((m,1))
    for i in range(m):
        p[i,0] = my_sequential(X[i], W1, b1, W2, b2)
    return(p)
```

We can try this routine on two examples:

```
X_tst = np.array([
    [200,13.9], # postive example
    [200,17]]) # negative example
X_tstn = norm_l(X_tst) # remember to normalize
predictions = my_predict(X_tstn, W1_tmp, b1_tmp, W2_tmp, b2_tmp)
```

To convert the probabilities to a decision, we apply a threshold:

```
yhat = np.zeros_like(predictions)
for i in range(len(predictions)):
    if predictions[i] >= 0.5:
        yhat[i] = 1
    else:
        yhat[i] = 0
print(f"decisions = \n{yhat}")
```

This can be accomplished more succinctly:

```
yhat = (predictions >= 0.5).astype(int)
print(f"decisions = \n{yhat}")
```

Network function¶

This graph shows the operation of the whole network and is identical to the Tensorflow result from the previous lab. The left graph is the raw output of the final layer represented by the blue shading. This is overlaid on the training data represented by the X's and O's.

The right graph is the output of the network after a decision threshold. The X's and O's here correspond to decisions made by the network.

```
netf= lambda x : my_predict(norm_l(x),W1_tmp, b1_tmp, W2_tmp, b2_tmp)
plt_network(X,Y,netf)
```

Done

Path to AGI

Neural Networks and Artificial General Intelligence (AGI)

1. Introduction

- The **dream of AGI (Artificial General Intelligence)**—building an AI system as intelligent as a human—has been a long-standing goal in AI research.
 - The timeline for achieving AGI is **uncertain**; it could take **decades, centuries, or even longer**.
 - There is **hype and misconceptions** around AGI, often due to confusion between **ANI (Artificial Narrow Intelligence)** and AGI.
-

2. ANI vs. AGI

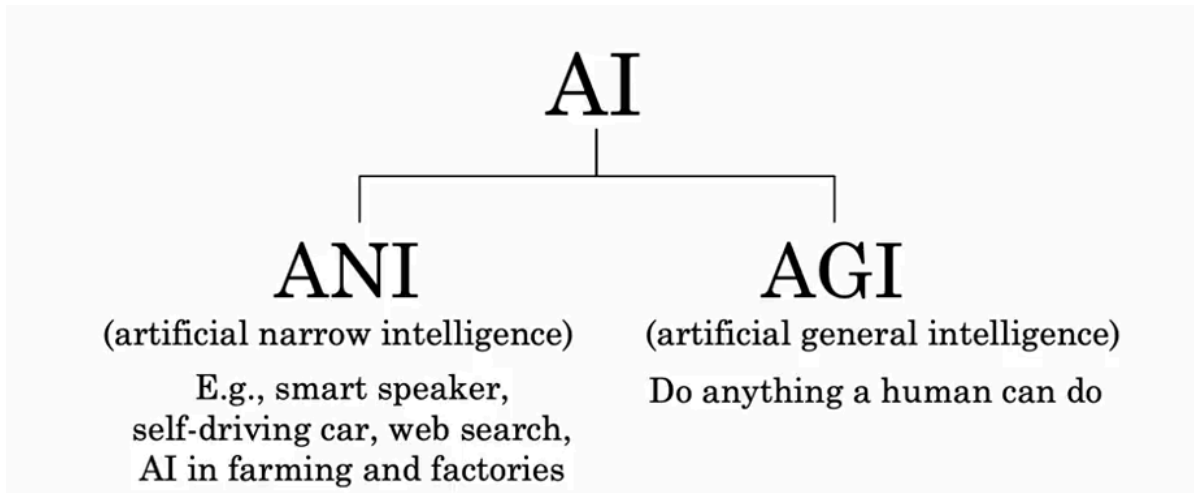
Artificial Narrow Intelligence (ANI)

- **Definition:** AI that is specialized in **one task** and can perform it very well.
- **Examples:**
 - Smart speakers (e.g., Alexa, Siri)
 - Self-driving cars
 - Web search engines
 - AI in **farming, factories, medical diagnosis**
- **Key Point:** ANI has made **significant progress** and is creating real-world value.

Artificial General Intelligence (AGI)

- **Definition:** AI that can perform **any intellectual task that a human can do**.
- **Challenge:** Despite rapid advances in ANI, **there has been little to no real progress towards AGI**.

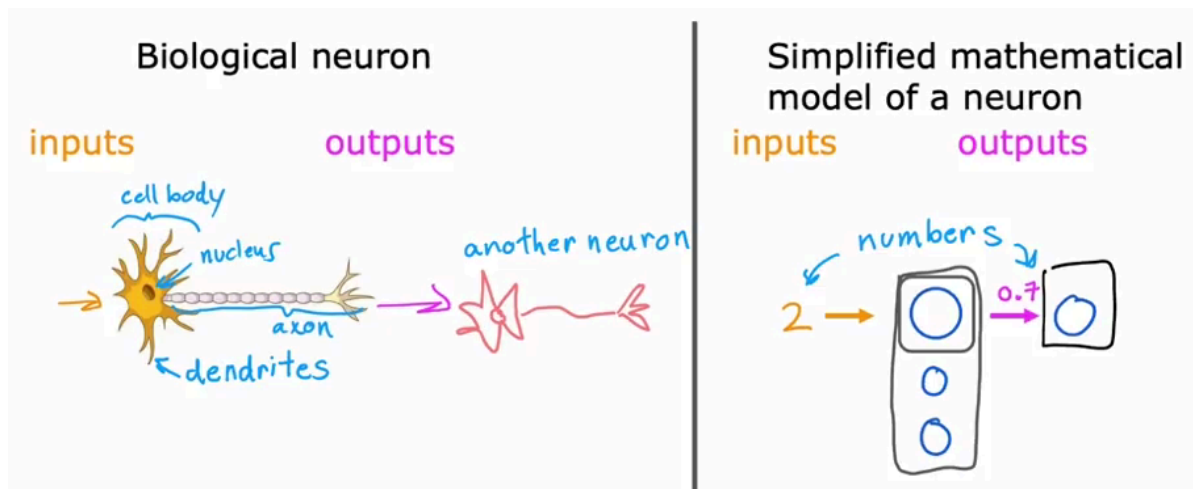
- **Misconception:**
 - Progress in **ANI (narrow AI) ≠ Progress in AGI.**
 - Many assume that because AI is improving, AGI is also advancing, which is **not necessarily true.**



3. Why Simulating the Brain Won't Easily Lead to AGI

- Early AI researchers believed that **simulating a large number of neurons** might lead to intelligence.
- **Why this approach is flawed:**
 1. **Artificial neural networks are too simple**
 - A **logistic regression unit (perceptron)** is **far simpler** than a **biological neuron**.
 - The brain's actual computation is **far more complex** than current artificial models.
 2. **We don't fully understand how the brain works**
 - Neuroscience still **lacks a complete understanding** of how neurons function.
 - **Unknown mechanisms** make it hard to replicate intelligence in AI.
- **Conclusion:**

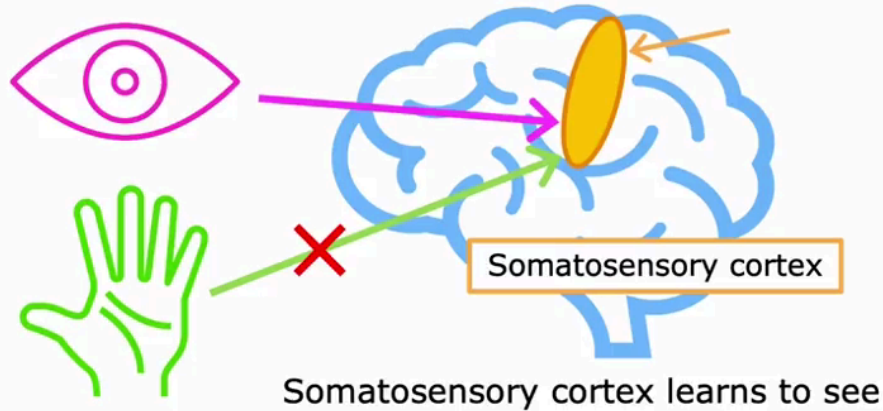
- Simulating a neural network **does not mean we are simulating intelligence.**
- Building AGI **requires more than just scaling up today's neural networks.**



4. The "One Learning Algorithm" Hypothesis

- **Idea:** The brain **might** operate using **a single or small set of learning algorithms.**
- **If true:** Discovering this algorithm could allow us to replicate human intelligence in AI.
- **Evidence from Neuroscience:**
 - The **same brain tissue** can learn **different tasks** based on input data.
 - Experiments show that different parts of the brain can **adapt** to new sensory inputs.

The "one learning algorithm" hypothesis



[Metin & Frost, 1989]

5. Neuroscientific Experiments Supporting Adaptability

Experiment 1: The Auditory Cortex Learning to See

- Normally, the **auditory cortex** processes **sound** from the ears.
- **Experiment:** If rewired to receive **visual input**, it **learns to see** instead of hear.

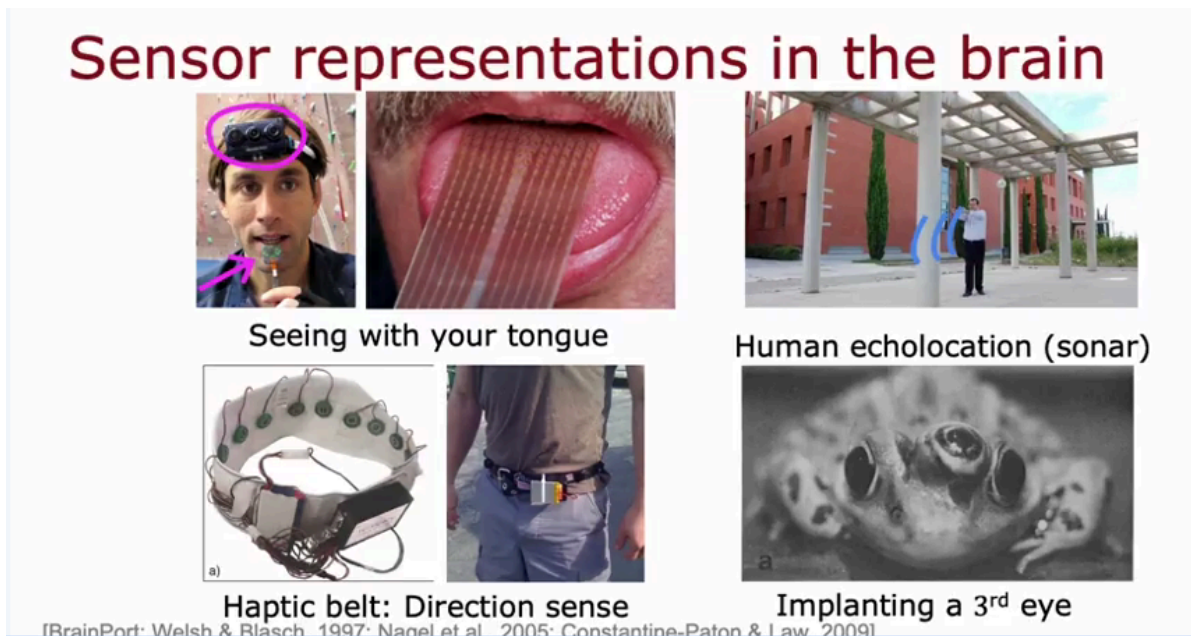
Experiment 2: The Somatosensory Cortex Learning Vision

- The **somatosensory cortex** (responsible for **touch processing**) was rewired to receive **visual data**.
- **Result:** It adapted and learned to process **visual information** instead of touch.

Experiment 3: Alternative Sensory Perception

- **People learning to "see" with their tongue:**
 - A **camera** mounted on a person's forehead converted images into **voltage patterns** applied to the tongue.
 - Over time, individuals **learned to perceive images** using their tongue.
- **Human Echolocation:**

- Some people have learned to **navigate environments** using **clicking sounds**, similar to bats and dolphins.
- **Magnetic Compass Integration:**
 - A ring of **buzzers on the waist** provided continuous feedback about **North**.
 - Users eventually **felt an innate sense of direction**.



Implication for AI

- The **brain is incredibly adaptable (neuroplasticity)**, meaning **one learning algorithm** may handle multiple tasks.
- **If** AGI researchers can find this **general learning algorithm**, it could be **implemented in AI**.

6. The Future of AGI Research

- **Challenges:**
 - We **don't yet know** if intelligence is based on **one core algorithm** or **multiple mechanisms**.

- Even if we discover it, we may **lack the computational power** to implement it effectively.
 - **Hope for the Future:**
 - Continued research may eventually lead to **approximating** a general intelligence model.
 - The **next breakthrough in AGI** may not come from **scaling up neural networks** but from **new fundamental discoveries**.
-

7. Neural Networks: Practical Use vs. AGI

- **Even without AGI**, modern **machine learning and neural networks** are **powerful tools**.
 - **Neural networks can still drive innovation** in:
 - Computer vision
 - Natural language processing (NLP)
 - Robotics
 - Healthcare
 - **Practical AI applications** will continue to provide **value** even without reaching AGI.
-

8. What's Next? Efficient Neural Network Implementations

- Next lessons will explore **optimizing neural networks**.
 - **Key topic: Vectorization for efficient computation** in neural networks.
-

Summary

- **ANI (narrow AI)** has made huge progress, but **AGI (general AI) is still far away**.
- Simply **scaling up** neural networks **won't** necessarily lead to AGI.

- **The brain is highly adaptable**, possibly using **one core learning algorithm**.
 - Neuroscientific experiments suggest **the same brain tissue** can learn **different types of tasks**.
 - **Future AGI research** may depend on **new discoveries** rather than just bigger neural networks.
 - **Even without AGI**, neural networks remain **highly useful in real-world applications**.
-

Vectorization in Neural Networks: Efficient Forward Propagation

1. Importance of Vectorization in Deep Learning

- Deep learning has **scaled up** due to **vectorized implementations** of neural networks.
 - **Vectorization** enables **efficient computation** using **matrix multiplications**.
 - **GPUs and modern CPUs** are optimized for large-scale **parallel computation**, especially for matrix operations.
 - **Without vectorization**, deep learning wouldn't have scaled as successfully as it has today.
-

2. Traditional (Non-Vectorized) Forward Propagation Implementation

- Previous implementation of **forward propagation** used **for-loops** for computation.
- Given:
 - **x** : Input features.
 - **w** : Weights of the neurons.

- **B**: Bias terms.
- **Loop-based computation:**
 - Iterates through **each neuron**.
 - Computes weighted sum ($Z = Wx + B$).
 - Applies activation function ($A = g(Z)$, where **g** is sigmoid).

3. Vectorized Implementation of Forward Propagation

Transforming Inputs into Matrices

- **Non-vectorized approach** processes one neuron at a time.
- **Vectorized approach** represents all neurons as **matrices**, eliminating the need for loops.
- Example:
 - **X as a matrix** (2D array in NumPy).
 - **W as a matrix** (weights stored in columns).
 - **B as a 2D array** (bias values as a row vector).

Replacing Loops with Matrix Multiplication

- In NumPy, matrix multiplication is performed using `np.matmul()` .

- **Equation:**

$$Z = XW + BZ = XW + B$$

- **Code Implementation:**

```
Z = np.matmul(X, W) + B # Computes weighted sum
A_out = sigmoid(Z)      # Applies activation function
return A_out
```

- **Advantages:**
 - Eliminates **slow for-loops**.

- Uses **highly optimized** matrix operations.
- Takes advantage of **parallelism in GPUs and CPUs**.

4. How Matrix Multiplication Improves Efficiency

- **Matrix multiplication** is a key operation in neural networks.
- `np.matmul()` efficiently multiplies large matrices **in parallel**.
- **All computations** (weights, inputs, activations) are handled as **matrices**.

5. Next Steps: Understanding Matrix Multiplication

- Next videos will cover **linear algebra concepts**:
 - **Vectors, matrices, transposes, and matrix multiplications.**
- If familiar with linear algebra, you can **skip to the final video** of the week.
- The **last video** will further explain how `matmul()` **enables vectorized forward propagation**.

For loops vs. vectorization

```
x = np.array([200, 17])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([-1, 1, 2])
```

```
def dense(a_in, W, b):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:, j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out
```

```
[1, 0, 1]
```

```
X = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
B = np.array([-1, 1, 2])
```

```
def dense(A_in, W, B):
    Z = np.matmul(A_in, W) + B
    A_out = g(Z)
    return A_out
```

```
[[1, 0, 1]]
```

Summary

- **Vectorization** is critical for scaling neural networks.

- **Matrix multiplication** (`np.matmul()`) replaces **for-loops**, making computations faster.
 - **Deep learning frameworks** (TensorFlow, PyTorch) use **vectorized implementations** for efficiency.
 - **Upcoming lessons** will explain **matrix multiplication** and its role in neural networks.
-

Matrix Multiplication for Neural Networks

1. Introduction to Matrices and Multiplication

- A **matrix** is a **2D array of numbers** (a grid of values).
 - **Matrix multiplication** is crucial for **vectorized implementations** of neural networks.
 - **Goal**: Understand how to multiply **vectors and matrices** step by step.
-

2. Dot Product of Two Vectors

- Given two vectors:

$$a = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, w = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

- **Dot product formula**:

$$z = (1 \times 3) + (2 \times 4) = 3 + 8 = 11 \quad z = (1 \times 3) + (2 \times 4) = 3 + 8 = 11$$

- **General formula**:

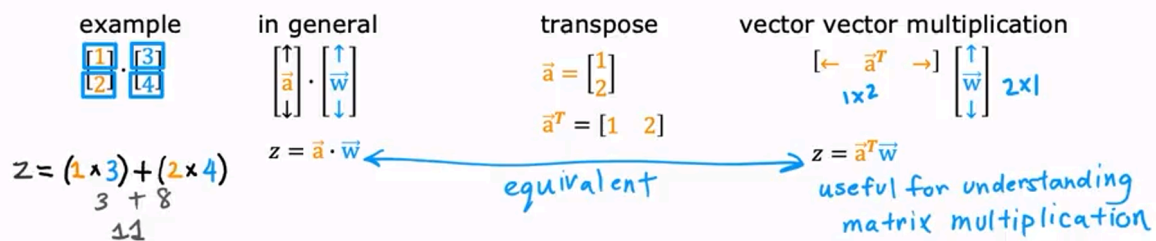
$$z = a_1 \cdot w_1 + a_2 \cdot w_2 + \dots + a_n \cdot w_n \quad z = a_1 \cdot w_1 + a_2 \cdot w_2 + \dots + a_n \cdot w_n$$

- The dot product sums the element-wise **products** of the two vectors.
-

3. Dot Product Using Transposition

- A **column vector** can be turned into a **row vector** using **transpose** (T):
 $a^T = [12]$
- The dot product can be written as **matrix multiplication**:
 $z = a^T w$
 - a^T is a $1 \times n$ **row vector**.
 - w is a $n \times 1$ **column vector**.
 - The result z is a **single number (scalar)**.
- **Conclusion:** Dot product is just **multiplication of a transposed vector with another vector**.

Dot products



4. Vector-Matrix Multiplication

- Multiplying a **vector by a matrix** extends dot products.
- Given:
 - **Vector**
 $a = [12]$
 - **Matrix**
 $W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix}$

Computation:

- **Step 1:** Transpose $\mathbf{a}$ to a **row vector**:

$$\mathbf{a}^T = [1 \ 2]$$

- **Step 2:** Compute :

$$\mathbf{Z} = \mathbf{a}^T \mathbf{W}$$

- First column of $\mathbf{W}$:

$$\begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

$$(1 \times 3) + (2 \times 4) = 3 + 8 = 11 \quad (1 \times 3) + (2 \times 4) = 3 + 8 = 11$$

- Second column of $\mathbf{W}$:

$$\begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

$$(1 \times 5) + (2 \times 6) = 5 + 12 = 17 \quad (1 \times 5) + (2 \times 6) = 5 + 12 = 17$$

- **Result:**

$$\mathbf{Z} = [11 \ 17]$$

- **Key Idea:** A **vector-matrix multiplication** produces a **new row vector**.
-

5. Matrix-Matrix Multiplication

- Instead of a single vector, we multiply **two matrices**.
- Given:

- **Matrix**

$$\mathbf{A} = \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix}$$

- **Matrix**

$$\mathbf{W} = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix}$$

Step 1: Compute $\mathbf{A}^T$ (Transpose of $\mathbf{A}$)

- Transpose $\mathbf{A}$ by **swapping rows and columns**:

$$\mathbf{A}^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \end{bmatrix}$$

Step 2: Compute $A^T W$

- Row 1:

- Dot product of with **each column of** W :

$$\begin{bmatrix} 1 & 2 \end{bmatrix}$$

- $(1 \times 3) + (2 \times 4) = 11$ $(1 \times 3) + (2 \times 4) = 11$
- $(1 \times 5) + (2 \times 6) = 17$ $(1 \times 5) + (2 \times 6) = 17$

- Row 2:

- Dot product of with **each column of** W :

$$\begin{bmatrix} -1 & -2 \end{bmatrix}$$

- $(-1 \times 3) + (-2 \times 4) = -11$ $(-1 \times 3) + (-2 \times 4) = -11$
- $(-1 \times 5) + (-2 \times 6) = -17$ $(-1 \times 5) + (-2 \times 6) = -17$

Final Result:

$$Z = A^T W = \begin{bmatrix} 11 & 17 \\ -11 & -17 \end{bmatrix}$$

6. Key Takeaways

- **Dot product:** Multiplication of two vectors, resulting in a **scalar**.
- **Vector-matrix multiplication:** Computes multiple dot products, resulting in a **vector**.
- **Matrix-matrix multiplication:** Computes **multiple vector-matrix multiplications**, producing another **matrix**.
- **Transposing matrices** allows for flexible multiplication.

7. Next Steps

- The **next video** will cover the **general rule of matrix-matrix multiplication**.
- Understanding this is crucial for **vectorizing** neural networks efficiently.

Summary

- **Dot product:** A sum of element-wise multiplications of two vectors.
 - **Vector-matrix multiplication:** Computing dot products of a vector with each **column of a matrix**.
 - **Matrix-matrix multiplication:** Computing multiple vector-matrix products.
 - **Matrix transposition** is used to arrange **rows and columns** for multiplication.
 - **Matrix operations** are fundamental to **neural networks** and **deep learning optimizations**.
-

Matrix Multiplication: General Form and Application in Neural Networks

1. Introduction to Matrix Multiplication

- **Matrix multiplication** is fundamental for **vectorized implementations** of neural networks.
 - The goal is to understand **how two matrices are multiplied** and why it is useful in **deep learning**.
-

2. Understanding Matrix Transpose (A^T)

- **Matrix A :** A 2×3 matrix (2 rows, 3 columns).

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix}$$

- **Transpose (A^T):** Swaps rows and columns.

$$A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix}$$

- **Interpretation:** The **columns** of A become the **rows** of A^T .
-

3. Matrix Multiplication: $A^T \times W$

- **Matrix W** : A 2×4 matrix (2 rows, 4 columns).

$W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 10 \end{bmatrix}$

- **Multiplication Rule:**

- The **number of columns in A^T** must match the **number of rows in W** .
- Since **A^T is 3×2** and **W is 2×4** , multiplication is **valid**.
- The result will be a **3×4 matrix (Z)**.

4. Computing Elements of Matrix $Z = A^T \times W$

Each element of Z is computed as a **dot product** between a **row of A^T** and a **column of W** .

Example 1: First Row, First Column ($Z[1,1]$)

- Compute the **dot product** of **first row of A^T** and **first column of W** :

$$(1 \times 3) + (2 \times 4) = 3 + 8 = 11 \quad (1 \times 3) + (2 \times 4) = 3 + 8 = 11$$

Example 2: Third Row, Second Column ($Z[3,2]$)

- Compute the **dot product** of **third row of A^T** and **second column of W** :

$$(0.1 \times 5) + (0.2 \times 6) = 0.5 + 1.2 = 1.7 \quad (0.1 \times 5) + (0.2 \times 6) = 0.5 + 1.2 = 1.7$$

Example 3: Second Row, Third Column ($Z[2,3]$)

- Compute the **dot product** of **second row of A^T** and **third column of W** :

$$(-1 \times 7) + (-2 \times 8) = -7 + (-16) = -23 \quad (-1 \times 7) + (-2 \times 8) = -7 + (-16) = -23$$

Final Matrix Z After Computation

$Z = \begin{bmatrix} 11 & 17 & 23 & 29 \\ -11 & -17 & -23 & -29 \\ 1.7 & 2.7 & 3.7 & 4.7 \end{bmatrix}$

5. Key Properties of Matrix Multiplication

1. Dimension Rule:

- Multiplication is only valid if the **number of columns** in the first matrix matches the **number of rows** in the second matrix.
- Example: If A^T is 3×2 and W is 2×4 , multiplication is **valid** and results in a 3×4 matrix.

2. Resulting Matrix Dimensions:

- The resulting matrix ($Z = A^T \times W$) will have:
 - **Rows** = Rows of A^T
 - **Columns** = Columns of W
 - Example: $A^T (3 \times 2) \times W (2 \times 4) \rightarrow Z (3 \times 4)$
-

6. Why Matrix Multiplication Matters in Neural Networks

- Each row in A^T represents activations from a previous layer.
 - Each column in W represents weights associated with neurons in the next layer.
 - Matrix multiplication enables **vectorized** implementations, making **forward propagation** extremely efficient.
-

7. Next Steps: Applying Matrix Multiplication to Neural Networks

- The next lesson will **apply matrix multiplication** to the **vectorized implementation of a neural network**.
 - Using NumPy's `np.matmul()`, we can replace **slow loops** with **fast matrix operations**.
 - **Vectorization** makes **neural networks** run **significantly faster**.
-

Summary

- **Matrix multiplication** follows the rule:
$$(m \times n) \times (n \times p) = (m \times p) (n \times n) \times (n \times p) = (m \times p)$$
 - **Each element of the resulting matrix is a dot product** of a row from the first matrix and a column from the second matrix.
 - **Matrix multiplication is essential for efficient neural network implementations.**
 - **Next lesson:** Applying matrix operations to **vectorized forward propagation**.
-

Vectorized Implementation of Forward Propagation in a Neural Network

1. Introduction to Vectorized Forward Propagation

- **Goal:** Implement forward propagation efficiently using **matrix multiplication**.
 - **Key concept:** Replace slow **for-loops** with NumPy's `matmul()` function for faster execution.
-

2. Computing $Z = A^T \times W + B$ in Code

- **Step 1:** Define **input feature matrix** `A` as a NumPy 2D array.
- **Step 2:** Compute the **transpose** (`A.T`) using NumPy's `.T` function.
- **Step 3:** Define **weight matrix** `W` and **bias matrix** `B`.
- **Step 4:** Compute **Z (weighted sum of inputs)** using `np.matmul()`.
- **Step 5:** Apply **activation function** (`sigmoid`) element-wise to `Z`.

Matrix multiplication in NumPy

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

```

A=np.array([[1,-1,0.1],
            [2,-2,0.2]])
W=np.array([[3,5,7,9],
            [4,6,8,0]])
Z = np.matmul(AT,W)
or
Z = AT @ W

AT=np.array([[1,2],
            [-1,-2],
            [0.1,0.2]])

AT=A.T
      ↳ transpose

result [[11,17,23,9],
        [-11,-17,-23,-9],
        [1.1,1.7,2.3,0.9]]

```

3. Matrix Representation in Forward Propagation

Example: Input, Weights, and Biases

Given:

- Input features (A^T):

$$AT = [217] A^T = \begin{bmatrix} 2 & 17 \end{bmatrix}$$

(Represents a single input example: **200-degree coffee roasted for 17 minutes.**)

- Weight matrix (W):

$$W = [w_{1,1} \ w_{1,2} \ w_{1,3} \ w_{2,1} \ w_{2,2} \ w_{2,3}] \quad W = \begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix}$$

(Each column represents the weights of a neuron.)

- Bias matrix (B):

$$B = [b_1 \ b_2 \ b_3] \quad B = \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix}$$

Computing $Z = A^T \times W + B$

Each element of z is computed using **dot products**:

$$Z_{1,1} = (2 \times w_{1,1}) + (17 \times w_{2,1}) + b_1 \quad Z_{1,1} = (2 \times w_{1,1}) + (17 \times w_{2,1}) + b_1$$

$$Z_{1,2} = (2 \times w_{1,2}) + (17 \times w_{2,2}) + b_2 \quad Z_{\{1,2\}} = (2 \times w_{\{1,2\}}) + (17 \times w_{\{2,2\}}) + b_2$$

$$Z_{1,3} = (2 \times w_{1,3}) + (17 \times w_{2,3}) + b_3 \quad Z_{\{1,3\}} = (2 \times w_{\{1,3\}}) + (17 \times w_{\{2,3\}}) + b_3$$

After computation:

$$Z = [165 \quad -531 \quad 900] \quad Z = \begin{bmatrix} 165 & -531 & 900 \end{bmatrix}$$

Step 6: Applying the Sigmoid Activation Function

- Apply **sigmoid function** to each element of Z :

$$A = g(Z) = \frac{1}{1 + e^{-Z}} \quad A = g(Z) = \frac{1}{1 + e^{-Z}}$$

- Results:

(Values are rounded due to numerical precision.)

$$A = [1 \quad 0 \quad 1] \quad A = \begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

4. Code Implementation in Python

Defining Forward Propagation Using NumPy

```
import numpy as np

# Input feature matrix (1×2)
A_T = np.array([[2, 17]])

# Weight matrix (2×3)
W = np.array([[w11, w12, w13],
              [w21, w22, w23]])

# Bias matrix (1×3)
B = np.array([b1, b2, b3])

# Compute Z (weighted sum)
```

```
Z = np.matmul(A_T, W) + B

# Apply sigmoid activation function
A_out = 1 / (1 + np.exp(-Z))

print(A_out) # Output activations
```

5. Alternative Implementation Using @ Operator

- Instead of `np.matmul()`, NumPy allows the `@` operator for matrix multiplication:

```
Z = A_T @ W + B
```

- **Both methods are equivalent**, but `np.matmul()` is **clearer and preferred** in this course.

6. TensorFlow Convention for Input Matrices

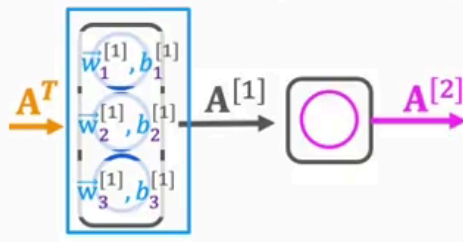
- **In TensorFlow**, examples are stored **in rows**, not in the **transpose format** (`AT`).
- Code implementation in TensorFlow follows:

where

`A_in` stores **input examples as rows**.

```
Z = np.matmul(A_in, W) + B
```

Dense layer vectorized



$$A^T = \begin{bmatrix} 200 & 17 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 & -3 & 5 \\ -2 & 4 & -6 \end{bmatrix}$$

$$B = \begin{bmatrix} -1 & 1 & 2 \end{bmatrix}$$

$$Z = A^T W + B$$

$$\begin{bmatrix} 165 & -531 & 900 \end{bmatrix}$$

$$z_1^{[1]} \quad z_2^{[1]} \quad z_3^{[1]}$$

$$A = g(Z)$$

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

```

A
AT = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([-1, 1, 2])
a_in
def dense(AT,W,b):
    z = np.matmul(AT,W) + b
    a_out = g(z)
    return a_out
a_in
[[1,0,1]]
    
```

7. Why Vectorization Matters?

- **Matrix operations are highly optimized** in modern CPUs and GPUs.
- **Replaces slow for-loops with fast batch processing.**
- **Leads to significant performance improvements** in training and inference.

8. What's Next? Training Neural Networks

- **Next topic:** Training neural networks using **backpropagation and optimization algorithms.**
- **Goal:** Learn how to **update weights (W)** and **biases (B)** to improve accuracy.

Summary

- **Vectorization** makes forward propagation **much faster** using **matrix multiplication.**
- NumPy's `matmul()` function efficiently computes $Z = A^T \times W + B$.
- **Applying the sigmoid function element-wise** gives activation outputs.

- **TensorFlow stores input data differently**, so input examples are laid out in **rows** instead of **columns**.
 - **Next step**: Learning how to train neural networks by adjusting weights and biases.
-
-